DIASPORUM

Full System Design — v10.11

Product scope • Architecture • Domain model • PostgreSQL • APIs • Realtime • Security • Operations

Document control	Value
Version	10.11
Date	26 October 2026
Status	Target architecture aligned to PostgreSQL schema v10.11 clean-install candidate; static consistency checks completed and DOCX/PDF render inspected; live database execution still required.
Baseline material	Full System Design v10.8 source consolidated into v10.9; Reels added in v10.10; Community Help & Offers added in v10.11; PostgreSQL Schema v10.11; v10.11 Change and Audit Report.
Runtime status	Design and schema candidate only; SQL has not been executed against a PostgreSQL server in this environment
Primary implementation	Spring Boot modular monolith, self-hosted PostgreSQL, Expo/React Native, Next.js, Ably, Cloudflare R2

Implementation warning
This document describes the intended design and implementation contracts. It does not claim that all features are implemented, deployed, security-audited or load-tested. Schema v10.11 is a clean-install candidate, not an in-place migration. The SQL has not been executed against a PostgreSQL server in this environment.

Contents

1. Executive summary and product principles
2. Product scope and release boundaries
3. Community model, country taxonomy and visibility
4. Roles, permissions and trust boundaries
5. UX and navigation principles
6. Target architecture and technology decisions
7. Core workflows and transaction boundaries
8. Feed and social content
9. People Connect and direct messaging
10. Business directory
11. Events
12. Jobs and employment listings
    12.4 Local service marketplace
13. Spare Bag parcel transport marketplace
14. Profile-based matchmaking
15. Community chat and persistent voice rooms
16. API design and endpoint catalogue
17. Ably realtime contract
18. PostgreSQL logical model and schema catalogue
19. Media and Cloudflare R2
20. Authentication, security and privacy
21. Background jobs, notifications and reliability
22. Deployment, observability and scaling
23. Testing and acceptance criteria
24. Delivery roadmap
25. Architecture decisions and open launch gates
    - Appendix A. Historical v10.8 schema inventory (superseded by Appendix K)
    - Appendix B. Historical lifecycle and API conventions (superseded where inconsistent with Appendix K)
    - Appendix C. Historical source artifacts and limits

- Appendix D. Historical v10.0 updates: marketplaces, visibility and billing
- Appendix E. Historical v10.2 follow-up decisions
- Appendix F. Historical v10.3 feed discovery, featured items and view tracking
- Appendix G. v10.5 schema update and change summary (historical baseline)
- Appendix H. v10.6 billing quota behaviour and deferred feature (carried forward)
- Appendix I. Historical v10.7/v10.8 service marketplace, messaging contexts and temporary voice comments (superseded where inconsistent with Appendix K)
- Appendix J. Historical v10.8 shared PostGIS distance and nearby discovery
- *Appendix K. v10.9 consolidated batch update: social graph, voice lifecycle, moderation, polls, featured placements, business accounts and push notifications*
- *Appendix L. v10.10 Reels short-video feature: feed modes, current-address Nearby sorting, media and publication integrity*

• *Appendix M. v10.11 Community Help & Offers: structured requests/offers, lifecycle and contextual responses*
Page numbers are intentionally omitted from this static contents list; Word navigation pane is enabled by the heading styles.

1. Executive summary and product principles

Diasporum is a mobile-first diaspora community and practical-support platform. It combines a country-aware social feed and Reels short-video experience, structured Community Help & Offers, people discovery, shared personal/business messaging, local business directory, events, jobs, Spare Bag parcel-transport matching, property and vehicle listings, an opt-in matchmaking experience and a lead-generation service marketplace. The v10.9 platform decisions and v10.10 Reels feature are carried forward in this v10.11 release; Appendix M defines Community Help & Offers as structured feed content with a dedicated lifecycle table and contextual messaging.

1.1 Non-negotiable principles

- **Backend is authoritative.** All access control, category validation, matching eligibility, age gating, capacity enforcement, media access and moderation decisions are enforced in Spring Boot and, where appropriate, PostgreSQL constraints/functions.
- **PostgreSQL is the durable source of truth.** Ably distributes realtime events but is not the message history store. R2 stores object bytes; PostgreSQL stores metadata and access policy.
- **Visibility is separate from classification.** A category explains what content is about. A scope determines who can discover it. Categories must never implicitly broaden an audience.
- **Least privilege.** OIDC identity establishes who the user is; application roles, membership and module-specific permissions determine what they can do.
- **Private data is not hidden only by the UI.** APIs must omit prohibited fields entirely, including original photo URLs for blurred/hidden matchmaking profiles and private address coordinates.
- **Reliable async delivery.** Persist durable state and outbox events in one database transaction; publish after commit with retries and idempotent consumers.
- **Modular monolith first.** Keep domain boundaries clear within Spring Boot. Introduce separately deployed services only where operational or scaling evidence justifies it.
- **Reviewable and reversible changes.** Use versioned migrations, clean staging installs, tested backups and release gates.

1.2 Product rules that shape the architecture

- Exactly one permanent General Chat Room per community. There is no permanent General Voice Room.
- Voice-room definitions persist independently from live sessions. A room remains discoverable when offline unless explicitly archived, locked or suspended; ending a session never expires the room. Live comments are session-scoped and are purged only after a session definitively ends or expires.
- Matchmaking is profile-based: users set preferences, like or pass profiles, and become matched when likes are mutual. Matched users can chat normally.
- Matchmaking is restricted to users aged 18 or older, with backend enforcement on all entry points and API reads/writes.
- Country practical categories are used by Feed, with optional country-topic links for Events. Jobs, Events, Business Directory, voice rooms, property and vehicle listings, and Spare Bag have their own taxonomies.
- Spare Bag connects sender requests with passenger offers; it does not by itself guarantee carriage, delivery, insurance, customs clearance, licensed transport, escrow or payment protection.

Release status
The v10.11 SQL schema is a static-reviewed candidate only and has not been executed against PostgreSQL in this environment. Treat this as an implementation baseline pending runtime schema install, migration rehearsal, integration, privacy and concurrency testing.

2. Product scope and release boundaries

Module	Primary user value	Core data/actions
Feed / Posts	Share practical and community information	Text, media, links, polls, Reels, Community Help REQUEST/OFFER posts, category badges, comments, reactions, saves, shares, reports
People Connect	Find and maintain relationships	Discover people, directional follows, derived mutual connections, blocks, privacy controls
Direct messaging	Private conversations	One shared context-aware conversation system; requests, business-side participants, durable messages, read cursors, attachments, message quotas and privacy checks
Business Directory	Find useful local services	Categories, locations, hours, media, reviews, saves, ownership and verification
Events	Discover and organise activity	Event-specific categories, time zone, RSVP, capacity, venue/address or online destination, reminders
Jobs	Find work and opportunities	Job taxonomy, source and employment details, location, application route, saves, internal applications
Spare Bag	Connect parcel senders with travellers	Airports, route/date matching, sender requests, passenger offers, contact requests, safety reports
Matchmaking	Discover compatible adults	Profile DRAFT/ACTIVE/PAUSED/SUSPENDED/DELETED; directional LIKE/PASS with ACTIVE/WITHDRAWN/REWOUND current states and append-only history; matches MATCHED/UNMATCHED/CLOSED/BLOCKED; mutual active LIKE creates one match and accepted chat.
Community chat / voice	Persistent voice rooms	Persistent public/private rooms with separate live sessions, session-scoped live comments, listener/speaker controls and moderation

Module	Primary user value	Core data/actions
Administration and trust	Keep service safe and reliable	Reports, moderation, audits, roles, verification, editorial controls, privacy requests
Service marketplace	Find and contact local service providers	Service categories, personal/business providers, one base location plus radius, job posts, quotes/responses, lead access, reviews, reports; no booking or service payment

2.1 Initial-release exclusions and deliberate limits

- No client-direct database access. Mobile and web clients call the Spring Boot API.
- No custom WebSocket backend when Ably provides the realtime transport.
- No voice recording by default. Recording would require separate consent, access, retention and legal design.
- No automatic guarantee that jobs, businesses or Spare Bag participants are trustworthy merely because an account exists.
- No unreviewed AI-generated legal, medical, immigration or financial guidance presented as authoritative advice.
- No payment/escrow commitment for Spare Bag or job listings unless explicitly designed as a separate product capability.
- No two voice SFU providers for the same session. One provider is selected per session behind an adapter.

3. Community model, country taxonomy and visibility

3.1 Community identity

A community is identified by `(origin_country_id, destination_country_id)`, for example Ethiopians in the UK. City is an optional profile/location filter, not a default top-level community dimension. The same destination-country practical categories may be reused across communities in that country.

- `profiles.origin_country_id` and `profiles.current_country_id` express the user's origin and current country.
- `communities` defines an origin/destination pair; uniqueness prevents duplicate communities for the same pair.
- `community_memberships` controls actual membership lifecycle and community roles. Profile country fields alone do not grant membership or posting rights.
- `user_settings.default_community_id` and `active_community_id` drive context selection. Backend must validate that selected communities are active and memberships remain valid.
- Global platform roles are separate from community-level roles. A community moderator does not automatically become a platform moderator.

3.2 Category design

Taxonomy	Used by	Examples
Country practical categories	Feed; optional Event topic link	Immigration, NHS/Healthcare, Education, Housing, Baby Care, Finance
Event categories	Events	Workshop, Cultural Event, Social Gathering, Networking, Family, Sports

Taxonomy	Used by	Examples
Job categories	Jobs	Warehousing & Logistics, Warehouse Operative, Construction Labourer, Community Worker, IT
Business categories	Business Directory	Solicitor, Accountant, Restaurant, Translator, Tutor
Voice-room categories	Persistent voice rooms	Social, New to Country, Careers, Culture & Language, Help & Advice
Spare Bag categories	Parcel listings	Documents, Clothing, Gifts and other policy-allowed parcel classes
Interests / languages	People discovery and matchmaking	Football, Technology, Amharic, Tigrinya, English

`country_category_modules` selectively maps a country category to FEED or EVENT. It must not be used as a job occupation taxonomy or business type catalog. Job categories can be multi-assigned where a listing legitimately spans more than one category; the primary category and country compatibility rules are validated for published listings.

3.3 Scope is separate from category

Scope	Meaning	Enforcement
COMMUNITY	Discoverable only within a selected origin/destination community.	Require a valid community context and active membership; verify the community destination/country agrees with the item.
COUNTRY	Discoverable across eligible communities within the destination-country market.	Explicit choice at creation; queries and response DTOs must apply country, status, privacy, block and moderation filters.

Feed posts, Events and Jobs use destination/current country as the content market. Property and vehicle listings use their explicit country and audience scope. Spare Bag uses the country of the origin/departure airport as the listing market. Category choice never broadens scope. Airport references remain in `airports` rather than being flattened into address records.

4. Roles, permissions and trust boundaries

Role / state	Scope	Key restrictions
MEMBER	Community	Can use allowed community features while active; cannot grant roles or moderate others.
TRUSTED_HELPER	Community	Narrowly defined helper abilities; does not imply platform-wide authority.
MODERATOR	Community	Moderates content and, in voice sessions, performs permitted room moderation actions.
COMMUNITY_ADMIN	Community	Manages community-scoped settings and roles within delegated policy; cannot assign platform roles.

Role / state	Scope	Key restrictions
PLATFORM_ADMIN	Platform	High-privilege configuration; protected by audit, least privilege and stronger authentication.
PLATFORM_MODERATOR	Platform	Cross-community moderation with scoped tooling and audit logs.
CONTENT_EDITOR	Platform	Editorial changes, taxonomies and approved content types within assigned scope; not automatically an admin.
SUPPORT_AGENT	Platform	Support workflows with least privilege; sensitive data access must be justified and audited.

4.1 Authorization rules

- Never trust `user_id`, `owner_id`, `role`, `is_verified` or `community_id` supplied by the client as proof of permission.
- Resolve identity from a validated OIDC principal mapped through `(issuer, subject)` in `external_identities`.
- Check object ownership and visibility for every read, update, delete, upload grant, message operation and report action.
- Apply blocks in both directions wherever relevant: People Connect, matchmaking, search, notifications, direct messaging and contact suggestions.
- Use immutable audit entries for role assignment/revocation, moderation action, verification decision and sensitive support access.
- Platform moderators may override user-level protections only through explicit emergency/admin paths that record the actor and reason.

5. UX and navigation principles

The mobile app is the primary experience; the Next.js site supports discoverability, selected public pages and the administration console. Keep the information architecture familiar, but expose country/community context clearly and avoid presenting every feature as a category-filtered feed.

Screen / surface	Core behaviour
Home / Feed	Community feed by default; category badges; create post; category and visibility selectors; saved content and share actions.
Discover People	Privacy-aware people cards; language/interests; follow/friend actions; matchmaking is a distinct opt-in feature.
Messages	Conversation list, message requests, unread indicators from server read cursors, attachment preview, retry/offline handling.
Community Chat	One permanent General Chat per community; history pagination; realtime stream; unread count; mute/report/message controls.

Screen / surface	Core behaviour
Voice Rooms	Discover active public rooms or enter authorised private rooms; show stage capacity separately from total listeners; hand raise, host/moderator actions and clear speaking state.
Jobs	Filter by category, employment type, schedule, workplace, location and source; distinguish internal apply from external link.
Matchmaking	Adult-gated onboarding, preferences, LIKE/PASS cards and explicit display-name/photo visibility settings. Mutual likes produce a match and standard chat.
Spare Bag	Separate sender request and passenger offer flows; airport/date/route comparison; contact request; protect private collection/handover details.
Admin console	Dedicated moderation/report queues, category/catalog management, role management and audit log.

5.1 Interaction and accessibility

- Provide explicit empty, loading, offline, permission-denied, removed-content and retry states.
- Use optimistic UI only where the operation can be rolled back cleanly; never treat optimistic state as authoritative for matching, capacity or moderation.
- Provide accessible touch targets, screen-reader labels, dynamic text sizing, keyboard support on web, and clear colour-independent status indicators.
- Keep moderation/report/block actions accessible from profile cards, message conversations, posts, voice rooms, job/business listings and Spare Bag entries.
- Show timezone alongside event times and make external application destinations clear before navigation.

6. Target architecture and technology decisions

6.1 Technology stack

Layer	Choice	Responsibility
Mobile app	React Native + Expo	Primary client; secure token storage; navigation; camera/media selection; push registration; Ably client subscription.
Web / admin	Next.js	Web experience and administration console; use secure server-side/BFF sessions where appropriate.
Backend	Spring Boot modular monolith	REST APIs, domain services, validation, transactions, authorization, jobs, outbox publisher and provider adapters.
Database	Self-hosted PostgreSQL 15+	Durable domain data, constraints, indexes, transactional updates, search, revision snapshots, sequences and audit metadata.
Identity	Keycloak recommended; Authentik alternative	OIDC sign-in, Authorization Code + PKCE, token issuance and identity lifecycle.

Layer	Choice	Responsibility
Realtime	Ably	Authorised subscriptions and low-latency event delivery; not durable storage.
Object storage	Cloudflare R2	Media bytes, thumbnails, blurred images and processed assets; access via scoped signed URLs/API grants.
Voice SFU	LiveKit or Cloudflare Realtime SFU	Low-latency audio. Hide provider detail behind adapter; choose one provider for each session.
Push	Expo Push backed by APNs/FCM	Notifications when realtime app delivery is unavailable or not relied upon.
Background work	Spring worker/scheduler + PostgreSQL outbox/queue records	Retries, expiry, push, media processing and lifecycle reconciliation.

6.2 Backend module boundaries

Module	Owned domain responsibilities
identity	OIDC subject mapping, profiles, age/date-of-birth handling, account lifecycle and user data requests
community	Countries, origin/destination communities, memberships, active context and community roles
taxonomy	Country topics, module mapping, interests, languages and feature-specific category catalogs
feed	Posts, post media, comments, reactions, polls, saves/shares, feed visibility and reports
social	Follow/friend graph, blocks, privacy and direct conversations/message requests
chat	General chat, direct messages, per-room/conversation sequence numbers and read cursors
voice	Voice room/session lifecycle, participants, seats, speaker requests, host transfer, provider credentials and moderation
directory	Business listings, branches, addresses, hours, media, reviews and verification
events	Event content, event categories, RSVP, capacity, venue/address, calendar/reminders and voice-event link
jobs	Job taxonomy, listing sources, employment details, apply routes, saved jobs, applications and expiry
spare_bag	Airport directory use, sender/passenger listings, route/date matching, contact requests and private handover locations
matchmaking	Eligibility, profiles, preference matching, display visibility, directional swipes, mutual matches, blocks/reports

Module	Owned domain responsibilities
realtime	Ably token issuance, event envelope, outbox publication, event deduplication and capability scope
media	R2 upload grants, validation, media processing, thumbnails, blurred renditions and access grants
notifications	In-app notification state, delivery preferences, device tokens, push retries and reminders
moderation/admin	Reports, audit log, restrictions, review queues, verification and platform settings
search	PostgreSQL search and visibility-aware discovery across supported modules

6.3 PostgreSQL deployment model

- Keep PostgreSQL on a private network; only the backend and tightly controlled operations tools can connect.
- Use a supported JDBC driver, connection pool limits, statement/transaction timeouts and migration tool such as Flyway.
- Use separate least-privilege roles for schema migrations and runtime queries. Runtime credentials should not own the schema.
- Enable encrypted transport where traffic crosses a trust boundary; store database secrets outside source control.
- Automate backups and point-in-time recovery where available, and perform restore tests to a separate host.
- Use query plans, slow-query logs, pool saturation and index-use metrics to guide later partitioning or read scaling.

7. Core workflows and transaction boundaries

7.1 Sign-in and profile bootstrap

1. The Expo app launches the system browser for OIDC Authorization Code + PKCE. The web experience uses an appropriately secured session/BFF pattern.
2. Spring Boot validates token signature, issuer, expiry, configured audience and claims/scopes.
3. Map the stable `(issuer, subject)` pair through `external_identities` to `profiles.id`; do not auto-link identities solely because emails match.
4. On onboarding, create/update profile, settings, privacy defaults and membership for the resolved community using an idempotent transaction.
5. On every request, evaluate account status, membership status, feature eligibility, object visibility and relevant block/moderation state.

7.2 Durable chat message and realtime event

1. Client submits a message to the REST API with an idempotency/client message key.
2. Spring Boot authorises membership/participants, validates content, assigns the next room/conversation sequence number and inserts the message.
3. The same database transaction records an outbox event describing the committed message.
4. After commit, a worker publishes the event to the authorised Ably channel and marks the outbox delivery state.

5. Consumers deduplicate by event ID/message ID and order by persisted sequence; after reconnect they fetch missed history through REST.

7.3 Mutual-like matchmaking

1. User activates a matchmaking profile after backend adult eligibility checks and sets discovery preferences.
2. Discovery filters active eligible adults and applies blocks, visibility, verification settings and preference compatibility in both directions.
3. Each user records a directional LIKE or PASS. `matchmaking_swipes` holds the current state; `matchmaking_swipe_history` records each swipe and any reversal as an append-only event.
4. Create one canonical pair match only when both current directional swipes are ACTIVE LIKE. In the same transaction create or reuse the pair's accepted GENERAL direct conversation; do not create follows or a duplicate friendship row. Show the Matched badge while the match remains active.
5. Before a mutual match occurs, a user can cancel their outgoing active LIKE. The backend marks it WITHDRAWN and records LIKE_CANCELLED atomically; a withdrawn LIKE must not create a match.
6. Rewind reverses only the actor's most recent eligible unreversed swipe (including an accidental PASS); the backend marks the current swipe REWOUND and records a REWOUND history event in the same transaction. Arbitrary older swipes cannot be restored.
7. On unmatch, close the matchmaking relationship, restrict future sending according to the match policy, and retain the GENERAL conversation and message history. Do not remove independent follows; the Matched badge disappears. Rematching after unmatch is unsupported by the current policy.

7.4 Voice room join and promotion

1. Client requests to join a room; backend checks room/session status, privacy, invite or discovery permission, account restriction and capacity.
2. Lock or otherwise serialise on the session row, create an active participant record, and enforce `max_participants` before issuing a short-lived RTC token.
3. Listeners request stage access through `voice_room_speaker_requests`; only the active host or an authorised moderator may respond.
4. Approval serialises against other approvals, checks a free stage seat, promotes the participant, assigns a seat and records a moderation/audit event.
5. Demotion releases the seat and returns the participant to listener role. Explicit departure sets `left_at`; a genuine rejoin creates a new participation record. Reconcile provider presence and apply the configured expiry/hostless rules.

8. Feed and social content

8.1 Feed post rules

- Post types include text, photo/video, link and poll content. Media remains in R2, while relational associations and metadata remain in PostgreSQL.
- Each post has an author, destination-country category, visibility scope and moderation/publication status.
- Category must be active for the relevant country and mapped to FEED. Scope must be validated separately.
- Community-scope queries require active membership and correct community context. Country-scope content is discoverable only in the relevant destination country and subject to all other policy filters.
- Comments/replies, reactions, polls/votes, saves, shares, editing/deletion and reports must enforce rate limits and ownership.
- Content removed or hidden by moderation must not leak through search, saved lists, media URLs, notification previews or cached feed pages.

8.2 Read models and pagination

Use stable cursor pagination (for example, `created_at` plus unique ID) rather than large offsets for growing feeds. When results are cached, include visibility context and authorization-affecting state in the cache design. Invalidate or recheck after block, membership, moderation or scope changes.

8.3 Example create-post contract

```
POST /api/v1/posts
Authorization: Bearer <OIDC access token>
Idempotency-Key: <opaque key>

{
  "communityId": "uuid",
  "categoryId": "uuid",
  "visibilityScope": "COMMUNITY",
  "postType": "GENERAL",
  "title": null,
  "body": "...",
  "mediaIds": ["uuid"]
}
```

Server rules: derive author from the validated principal; confirm membership and account state; check country/category/module compatibility; verify media ownership and READY state; persist post and outbox event atomically.

8.4 Mixed-feed ordering and view tracking

The default For You feed is a mixed community timeline with a view-aware ordering rule: first eligible content the user has not meaningfully viewed, newest first within that group; then previously viewed eligible content, also newest first. Apply hard eligibility filters first, including country/community scope, membership, blocks, moderation, publication state, expiry/availability and dismissed content. Do not use view status to bypass relevance, safety or content lifecycle rules.

Use `user_content_views`, keyed by user, content type and content ID, for first/last meaningful view time and an optional qualified view count. A card qualifies when at least 50% of its visible area remains in the viewport for approximately one second, or when the user opens its full detail/thread view. Do not mark an item viewed because an API returned it, because it was preloaded, or because it was rendered off-screen. The thresholds are configurable client rules and should be tested on mobile and web.

The view-tracking endpoint should be idempotent and batch-capable. Upsert one row per user/content pair; preserve the earliest first_viewed_at, advance last_viewed_at monotonically, and increment view_count only for a qualified view event after a server-enforced or client-assisted cooldown. Preserve stable cursor pagination and de-duplicate candidate IDs across organic and featured pools. Content may reappear after meaningful updates, such as a new reply, event time/location change or job update; avoid repeatedly resurfacing unchanged content.

If a separate Latest mode is offered, it should remain reverse chronological among eligible content rather than being reordered by view status. View-aware ordering belongs to For You/default feed. The horizontal category tabs remain a separate navigation layer into focused Events, Jobs, Property, Vehicles, Business, Polls and Spare Bag experiences.

8.5 Featured content and feed interleaving

`featured_items` is a placement/scheduling record, not a replacement for the event, job, property, vehicle, business, Spare Bag or feed-post record itself. It references a target by `content_type` plus `content_id`, and records editorial versus paid-promotion kind, placement key, priority, status, start/end window, campaign key and audit actors. `featured_item_impressions` is an optional analytics stream for qualified exposure and position; it is separate from `user_content_views`, which supports user-level unseen/seen decisions.

Because one PostgreSQL foreign key cannot target different tables based on a discriminator, the polymorphic target reference has no direct FK to the domain table. The backend must validate that the target exists, the type matches, the target is published/eligible, and the user is allowed to see it. It must also recheck country/community scope, moderation, availability and expiry when serving the feed. When the target is removed or made ineligible, end/cancel its placement. A payment or promotion must not broaden content visibility or bypass moderation. v10.9 adds `featured_placement_rules`, an allow-list of content-type/placement combinations. The API chooses a placement based on product, entitlement/payment, status, audience and moderation; the client cannot claim arbitrary placement. Represent multiple surfaces as separate `featured_items` rows.

Interleave eligible featured cards into the organic timeline; do not force a featured card into every page or slot. Initial experiment: consider the first featured item after roughly 3-5 organic items, subsequent featured items after roughly 5-8 items when useful candidates exist, and keep paid placements at no more than roughly one per 8-10 feed items near launch. Treat these as configurable experiment defaults, not hard-coded product promises. Clearly label paid content as Sponsored/Promoted, apply tighter frequency caps near the top, and log qualified impressions/clicks for later tuning.

Apply content-type-specific relevance: events should reflect start date, attendance feasibility and capacity; jobs should reflect active status, freshness, location/work mode and user preferences where available; property/vehicle listings should reflect published status, location/price preferences and availability; business and Spare Bag items should use their own scope and relevance rules. Featured and paid status is only a boost within eligible candidates; it must not automatically outrank substantially more relevant content. The dedicated category tabs remain the complete search/browse experiences. Feed chronology should use a consistent effective publication timestamp from the canonical item (for example, published_at when available, otherwise the approved/created timestamp), not featured_items.starts_at. Event urgency is a separate relevance signal.

9. People Connect and direct messaging

9.1 Relationships and profile discovery

- Following is directional and stored in `user_follows`. A mutual connection is derived from reciprocal follow edges through `user_mutual_connections`; there are no separate `friend_requests` or `friendships` tables in v10.9. Prevent self-follow, require active profiles, respect `allow_followers`, and block follows in either direction when a block exists.
- People discovery may use controlled interests and languages, user preferences and optional city/online status, but must respect privacy and blocks.
- Profile visibility settings govern discoverability and displayed attributes. Do not treat a discovered profile as permission to message.
- Use `interests` and `languages` as controlled catalogs; `user_interests` and `user_languages` are mapping tables with one row per user/value.
- Blocks are directional records, but the enforcement effect is reciprocal: either user's block prevents unwanted discovery/contact between the pair.

9.2 Direct message lifecycle

Direct messaging is one shared system for personal users and Business Accounts. Relationship-based GENERAL conversations use the recipient’s privacy policy; context-based enquiries are linked to a specific item and may start as accepted conversations after all context/privacy/block/contact/billing checks. A pending stranger message request may contain exactly one initial message; ordinary follow-up messaging begins only after acceptance. A mutual matchmaking match creates/reuses the normal GENERAL conversation and a match-origin friendship.

- `direct_conversations` stores the canonical participant identities, optional Business Account identities on either side, a context type/resource ID and inbox activity fields. Its uniqueness key includes both participant identities and the context, so the same two parties have separate conversations for different listings or jobs but reuse the thread for the same context.
- `direct_conversation_participants` stores per-user inbox/read/archive/mute state for each side. For a Business Account side, active accepted members are associated with that side; authorization is rechecked on every request and business membership changes update conversation access.

- `direct_messages.sequence_no` is allocated monotonically within its conversation. The sender identity records both the acting user and optional Business Account identity. For a REQUESTED GENERAL conversation, only one initial message linked to the pending request may be sent until it is accepted.
- The last-message timestamp and preview must stay consistent when messages are created, and when the latest message is edited or soft-deleted. Reopening a context reuses the matching conversation; a different property, vehicle, business profile, event, job, Spare Bag contact or service request gets its own thread.
- Only authorised personal participants or active members acting for a participating Business Account may read, subscribe, send, mark read or access attachment grants. Blocks always override all context and relationship rules; privacy settings, context visibility, lead access, rate limits and business-role permissions are checked on every request.

9.3 Context-specific conversations and business participants

Use these eight direct-message contexts: `GENERAL`, `PROPERTY_LISTING`, `VEHICLE_LISTING`, `BUSINESS_PROFILE`, `EVENT`, `JOB_LISTING`, `SPARE_BAG_CONTACT` and `SERVICE_REQUEST`. `FEED_POST` is not included. Matchmaking is not a separate context: it uses GENERAL and a derived Matched badge.

A conversation is unique to the ordered participant identities plus context type and context ID. The same two users discussing Property X and Property Y have two conversations. Reopening Property X reuses the existing Property X thread. GENERAL has a null context ID and can exist independently of every contextual thread. Context IDs are polymorphic and must be checked against the correct resource table; the backend also verifies visibility, ownership/eligibility, privacy and blocks.

A published property, vehicle, business profile, event, job or eligible service request can initiate a context-specific conversation without a separate request-acceptance step, but only after its contact policy and other protections pass. For service requests, in-app chat requires an active `SERVICE_LEAD_ACCESS` row with `chat_enabled=true`; exposing phone/contact data requires `contact_details_unlocked=true`.

For GENERAL contact, mutual followers may qualify under `dm_permission='MUTUAL_FOLLOWERS'`; the backend still evaluates the recipient’s privacy policy, blocks and message-request rules. A one-way follow does not by itself grant messaging access. If an initial request is allowed but direct messaging is not, create a REQUESTED conversation and a message request with exactly one initial message. Recipients can accept/decline; accepted status enables subsequent messaging.

Business-to-user, user-to-business and business-to-business chats use the same tables. A Business Account is an endpoint identity, and an authorised active member is the acting sender. Membership revocation removes that user’s access without deleting the conversation or messages. A business-side chat cannot be accessed merely by knowing the business account ID.

When a matchmaking match becomes active, create/reuse an accepted GENERAL conversation; do not create follows or friendship records. The chat UI derives the Matched badge from the active match record. On unmatch, close the match and retain the conversation history while denying future sending according to policy. Independent follow edges are unchanged.

9.4 Read cursor and unread state

Field	Meaning / rule
`direct_conversations.last_message_seq`	Latest committed sequence allocated for that conversation.
`direct_messages.sequence_no`	Server-assigned unique order within its parent conversation.
`direct_conversation_participants.last_read_seq`	Last sequence that a participant has actually read; it cannot move backwards or exceed the assigned last sequence.
`chat_rooms.last_message_seq` / `chat_messages.sequence_no`	Equivalent room-scoped sequence mechanism for community chat.
`chat_room_user_state.last_read_seq`	Per-user, per-room read cursor for unread calculations.

Read state is a cursor per user per conversation/room—not a row per message read. Per-message read receipts can be added only if the product later needs “seen by each member” functionality.

10. Business directory

Business listings are country-aware directory entities, not feed posts. Business categories are separate from country practical topics and job categories. Listings may be country-wide and filtered by city, service, category and location.

- `business_listings` owns the business identity, description, contacts, status and verification lifecycle.
- `business_locations` links a business to one or more branches; each branch references `addresses`. Opening hours are associated with the relevant location so separate branches can have different hours.
- Use `business_categories`, `business_media`, `business_reviews` and `business_saves` for catalogue, photos, user review and bookmarks.
- Each user may submit at most one current review per business; review editing/deletion and reporting must be supported.
- Owners can manage only their own business or a branch they are authorised to manage. Owners cannot self-verify and cannot alter a review written by another user.
- Call links use validated `tel:` handling, websites are validated HTTP(S), and directions open a selected map application. Verification claims require approved status.
- Rating aggregates should be maintained transactionally or safely recomputed and periodically reconciled.

11. Events

- Events have a destination country, event-specific category, title/description, organiser, start/end times and required IANA timezone identifier.
- Event mode is `IN_PERSON`, `ONLINE_VOICE` or `ONLINE_EXTERNAL`. In-person events use a venue name plus shared `address_id` where a structured address is available; city/venue fallback remains possible.
- Online external events require a validated HTTP(S) URL. Online voice events link to a Diasporum room reference, not directly to provider credentials.
- `event_categories` is the primary classification. An optional country practical category can identify a topic such as Education or Healthcare; it does not replace the event category.
- `event_rsvps` tracks Going, Interested, Not Going and Waitlisted; `event_saves` and `event_media` support engagement.
- If capacity is enabled, RSVP acceptance/waitlist transitions must be protected by a transaction or row lock to avoid exceeding capacity.
- Events remain as historical records after an associated voice session ends. Event cancellation/changes trigger notifications and calendar updates where supported.

12. Jobs and employment listings

`job_listings` is a structured job entity capable of representing warehouse work and other external-employer roles as well as community-shared jobs and opportunities from community organisations. A job is not merely a feed post carrying a “Jobs” badge.

12.1 Separate concepts

Concept	Representation	Examples
Occupation / industry	`job_categories` and `job_listing_categories`	Warehousing & Logistics; Warehouse Operative; Construction Labourer; Community Worker; IT Support

Concept	Representation	Examples
Employment arrangement	`employment_type`	Permanent, fixed-term, contract, temporary, casual, zero-hours, seasonal, apprenticeship, internship, volunteer, freelance, commission, self-employed, other/unspecified
Work schedule / hours pattern	`work_hours_type` and `work_schedule`	Hours pattern: full-time, part-time, variable, shift-based, flexible or unspecified; schedule: day, night, rotating, weekends, mixed or other.
Workplace type	`ONSITE`, `HYBRID`, `REMOTE`, `MULTIPLE_LOCATIONS`, `FIELD_BASED`	Warehouse onsite; hybrid office role; remote customer support; multi-site or field work
Listing source	`source_type`	Employer direct, recruiter, community-shared, community organisation, external aggregator, platform editor
Application route	`application_method`	`INTERNAL` or `EXTERNAL`; independent of the source type
Visibility scope	`COMMUNITY` or `COUNTRY`	Community-level opportunity or explicitly country-wide listing

12.2 Listing and application controls

- External job listings may link to an employer ATS, employer website, recruiter portal or approved external destination. Validate HTTP(S) URLs and do not allow unsafe schemes.
- Community-shared postings must identify an accountable posting user and represent the source clearly. A source type never automatically proves legitimacy.
- Aggregated listings need source metadata and must be governed by permitted reuse/attribution rules. Expired source listings must not remain discoverable as open jobs.
- Published listings need an accountable posting user, active primary job category and valid country/scope/category assignments.
- Employer identity, optional business link, salary range/currency, workplace address or city, close date and media are represented when known. Keep city-only or remote jobs possible without inventing a street address.
- Internal applications are allowed only for published INTERNAL jobs. Do not store application records for EXTERNAL redirect-only listings.
- Internal applications include applicant, resume/media where supported, optional cover letter, status and timestamps. Employers cannot apply to their own vacancy.
- Employer/business owners manage only listings they own unless granted an explicit moderator/editor role. Reports/scam signals may pause or hide a listing during review.

12.3 Discovery

Search and filters should include job category, employment type, work schedule, workplace type, country/city, salary when provided, source type, employer and close date. Country practical categories such as NHS or Immigration are not the job taxonomy; jobs use their own category catalog.

12.4 Local service marketplace

The service marketplace connects customers with independent personal or business service providers. It is a lead-generation and contact product, not a booking, checkout or job-completion platform. Customers publish service requests; providers discover relevant requests, send a response/quote and, after gaining lead access, can reveal permitted contact details or start in-app chat. Actual work, appointments and payment are arranged off-platform.

12.4.1 Provider profiles and discovery

`service_providers` has exactly one owner: a personal profile or a Business Account. Each provider chooses service categories and maintains one editable base location plus `radius_km`; no separate provider-location or coverage-area tables are introduced. Provider discovery uses the base point and the request point, a PostGIS geography index, `ST_DWithin` radius matching and `ST_Distance` ordering. Providers can filter by category, budget, date, country/city and distance. A missing point must not be interpreted as global coverage.

12.4.2 Customer requests and provider responses

A request contains a category, title, description, country, approximate/service-area geography point and optional address reference, budget range/currency, preferred date, media, expiry and lifecycle status. Exact private address/contact information is not included in discovery payloads. Responses support interest, contact and quotes; a provider can maintain one response per request and revise it through the supported response lifecycle. There is no appointment/booking state machine.

12.4.3 Billing and lead access

Use the shared entitlement engine for `SERVICE_JOB_POST` and `SERVICE_LEAD_ACCESS`. Publishing a request is controlled by the configured customer price and quota. Lead access controls contact-detail unlock and/or initiating in-app chat. One provider/request pair has one idempotent access record: chat and contact details can be enabled together or upgraded later without another charge. Access grant and the corresponding `billing_usage_records` usage unit must commit atomically with a deterministic idempotency key. `SERVICE_QUOTE_SEND` remains available as a future-compatible, disabled-by-default feature; provider registration is not a separately billable feature.

12.4.4 Reviews, reporting and boundaries

Customers can review a provider after a recorded interaction (lead access or a response); because the platform has no booking or proof of completion, the product must not call these “verified completed-work” reviews. Users can report requests, provider profiles and reviews. Moderation outcomes are recorded separately from report submissions. No service booking, appointment scheduling, provider checkout, escrow or on-platform payment for the actual service is in scope.

13. Spare Bag parcel transport marketplace

Spare Bag connects a sender who wants a parcel carried with a passenger who has spare luggage capacity on a planned route. Both types are first-class structured listings with route and date matching; neither is simply a Feed post.

13.1 Listing types and lifecycle

Type	Meaning	Typical fields
SENDER_REQUEST	Owner needs a parcel transported.	Origin/destination airport, date window, parcel count/estimated weight, contents class, proposed fee, collection/handover arrangement.
PASSENGER_OFFER	Owner offers spare capacity on a planned trip.	Origin/destination airport, date window, available kg/count, maximum parcel weight, fee, handover options.

Listing lifecycle: DRAFT → optional PENDING_REVIEW → PUBLISHED → PAUSED/MATCHED/CLOSED/EXPIRED/REMOVED. Only published eligible listings appear in discovery. Workers expire stale listings and contact requests, while retaining moderation/audit history according to retention policy.

13.2 Airport, address and matching rules

- Use `airports` as the authoritative airport catalog with IATA code, name, city, country, timezone and optional coordinates.
- Match only compatible sender/passenger listing types with the same origin and destination airports and overlapping travel-date windows.
- Use the origin airport country for Spare Bag listing market scope. A community-scoped listing must use a compatible community whose destination country matches that market.
- Shared `addresses` can represent optional collection/handover locations. Precise/private locations must not appear in public listing DTOs; reveal them only to specifically authorised parties at an approved workflow stage.
- Validate price amount, currency, per-kg/per-parcel unit, requested weight/count, available capacity and maximum single-parcel weight.

13.3 Safety and limitations

- Display a prohibited/restricted-item policy, truthful listing warnings and route/immigration/customs safety messaging.
- Provide reporting and moderation for users, listings and contact requests; guard against scams, suspicious payment requests and harassment.
- Accepting a contact request is not a guarantee of carriage, delivery, insurance, customs clearance, licensed carriage, escrow or compensation.
- Do not expose private phone numbers or exact addresses merely because a listing is visible. Contact-request state determines authorised next steps.

14. Profile-based matchmaking

New v9 design

This model replaces the previous anonymous matchmaking request/candidate/interest/reveal-consent workflow. Matchmaking uses opt-in profiles, preferences, LIKE/PASS decisions and reciprocal likes. A successful match enables ordinary direct chat; there is no separate reveal-consent step.

14.1 Main records

Table	Responsibility
`matchmaking_profiles`	One opt-in matchmaking profile per user; profile attributes, status, verification state and name/photo visibility settings.
`matchmaking_preferences`	One preference record per user: age, distance, verified-only, children preferences and multi-select criteria.
`matchmaking_preference_genders`	User-selected genders eligible for discovery.
`matchmaking_preference_religions`	User-selected religion preferences.
`matchmaking_preference_languages`	User-selected language preferences.
`matchmaking_swipes`	Current directional LIKE/PASS decision and ACTIVE/WITHDRAWN/REWOUND state; current state is paired with append-only history events.
`matchmaking_swipe_history`	Append-only SWIPE, LIKE_CANCELLED and REWOUND events; a single-reversal index prevents reversing the same event twice.

Table	Responsibility
`matchmaking_matches`	Canonical unordered user pair; MATCHED/UNMATCHED/CLOSED/BLOCKED lifecycle and close metadata. Unmatch closes the match, preserves chat history and independent follows, and does not create or remove generic social relationships; rematching after unmatch is unsupported.
`matchmaking_genders`, `matchmaking_religions`	Controlled lookup catalogs, with policy-managed active values.

14.2 Profile attributes

Attribute group	Required attributes to model
Age and identity	Age calculated from `profiles.date_of_birth`; gender; nationality. Do not store a redundant age value that becomes stale.
Location	Address/reference suitable for private distance calculation. Exact address and coordinates must never be exposed by discovery.
Beliefs and education	Religion, education level, occupation, religion importance.
Relationship	Relationship intention, marital status, marriage timeline, long-distance relationship, willingness to relocate.
Family	Has children; wants children.
Lifestyle	Smoking, drinking, activity level.
Compatibility	Interests, languages, height in cm.

Interests and languages should reuse the controlled `interests`/`user_interests` and `languages`/`user_languages` structures where suitable. Multi-valued fields should not be encoded as comma-separated text. Personal attributes should have explicit visibility and retention treatment; collecting a field is not the same as displaying it.

14.3 Discovery preferences

Preference	Evaluation rule
`interested_in_gender`	Candidate's permitted matchmaking gender must be in the user's selected set, unless the product explicitly defines “any”.
`min_age` / `max_age`	Candidate must meet inclusive age range, with age calculated on the server from date of birth.
`max_distance_km`	Compute distance from protected coordinates/address data; do not return exact coordinates to the candidate.
`show_verified_only`	If enabled, candidate must meet the product's defined profile-verification threshold.
`has_children_preference` / `wants_children_preference`	Apply defined tri-state semantics: ANY, YES or NO; missing/unknown must not silently be treated as a match.

Preference	Evaluation rule
`religion_preferences`	Use controlled religion IDs and explicit “any” semantics for an empty selection.
`language_preferences`	Match against controlled user-language membership and configured proficiency requirements if the product defines them.

The backend applies preferences in both directions. User A fitting User B’s criteria is not sufficient if User B does not fit User A’s criteria. Product policy must define whether all criteria are hard filters or whether some are soft ranking signals; a hard filter cannot be silently converted into a ranking preference.

14.4 LIKE / PASS / mutual match flow

1. User creates and activates a matchmaking profile and supplies a valid birth date and required eligibility confirmation/verification state.
2. Discovery calculates candidates from active eligible adult profiles and excludes self, blocked pairs, suspended/deleted/paused profiles, already-passed profiles and users not mutually preference-compatible.
3. User selects LIKE or PASS. The backend records the current directional decision and an append-only `matchmaking_swipe_history` event; it rate-limits repeated or automated swiping.
4. On reciprocal ACTIVE LIKE, the backend creates one canonical match and creates or activates one accepted `direct_conversations` row in a transaction or equivalent idempotent workflow.
5. Before a mutual match, the sender may cancel an outgoing LIKE. Rewind may undo only the latest eligible unreversed swipe. Both actions write the current state and history atomically and are checked against blocks and matches.
6. Either participant can unmatch. The backend marks the match UNMATCHED with closed_at/closed_by/close_reason, closes or disables the linked GENERAL conversation, preserves its history, leaves independent follows untouched, and removes the Matched badge. Rematching after unmatch is not supported.

14.5 Name and photo visibility

Setting	Value	Response behaviour
Name visibility	VISIBLE	Return display name if the rest of the profile policy allows it.
Name visibility	BLURRED	Return a masked/anonymised representation, not the unmasked name in another DTO field.
Name visibility	HIDDEN	Omit the actual name; do not include it in hidden metadata or analytics payloads.
Photo visibility	VISIBLE	Return an authorised original/normal rendition URL.
Photo visibility	BLURRED	Return a server-generated blurred rendition only. Do not return the original object key/URL.
Photo visibility	HIDDEN	Do not return a photo URL, media ID or original path that reveals the protected photo.

14.6 Adult-only eligibility

- Only users aged 18 or older may enter or use matchmaking. Check date of birth server-side on activation, discovery, profile read, preference updates, swipes, match access, chat entry where tied to matchmaking, and any related notification/deep-link endpoint.
- Deny missing DOB, under-18 users, deleted/suspended users and accounts that fail the configured eligibility/verification process.
- `age_eligibility_confirmed_at` is a record of confirmation, not proof by itself. If the product requires stronger assurance, implement a separate verification state/process.
- Never rely on client routing or hidden navigation to enforce the age gate.

14.7 Report and block

- Integrate matchmaking profile/match report targets with the shared report queue; preserve sufficient context for moderator review without disclosing the reporter to the reported user.
- Integrate `user_blocks` across discovery, swipes, match reads, conversation history/send, search and notifications.
- Blocking either side must prevent renewed discovery/contact. Close or mark the existing match and apply the configured direct-conversation restriction; retain minimal moderation/audit context according to policy.
- Allow reporting without requiring the reporter to unblock or keep communicating with the reported user.
- Rate-limit likes, passes, profile activation and report submission; monitor abuse patterns without exposing sensitive preference data.

15. Community chat and persistent voice rooms

15.1 General Chat

- Exactly one permanent `GENERAL` text chat room per community, ensured through the unique partial index and idempotent seed/upsert process.
- Persist messages in PostgreSQL. Ably delivers new/update/delete events. Fetch history over REST with sequence/cursor pagination.
- `chat_rooms.last_message_seq` and `chat_messages.sequence_no` define room order. `chat_room_user_state.last_read_seq` stores each user's read cursor.
- Messages support reply-to-message, @mentions, optional media, moderation removal, message reports and rate limits. A dedicated restriction model distinguishes room/community CHAT mute and ban, with reason, expiry, revocation and an audit trail. `notifications_muted_until` remains a user notification preference, not a moderation control.
- No permanent General Voice Room is created or seeded.

15.2 Persistent room and disposable session model

Record	Responsibility
`voice_rooms`	Room metadata, owner/creator, category, public/private visibility, status, discoverability, `max_speakers`, `max_participants`, inactivity timeout and lifecycle timestamps.
`voice_room_sessions`	One live provider session/lifecycle instance; selected provider, provider room reference, status, hostless/grace expiry and lifecycle timestamps.
`voice_room_participants`	Per-join participation record, role, `joined_at`, `left_at`, connection state and presence heartbeat. Leave/rejoin create separate historical rows.
`voice_room_seats`	Numbered stage positions, each AVAILABLE/OCCUPIED/LOCKED, capped by `max_speakers`.

Record	Responsibility
`voice_room_speaker_requests`	Listener's pending request and host/moderator response state.
`voice_room_invites`	Private-room invitations, expiry and response state.
`voice_room_moderation_actions`	Actor, target, action and metadata for auditable moderation transitions.
`voice_room_messages`	Ephemeral session-scoped comments; delete on ENDED/EXPIRED, retain through recoverable INACTIVE grace period.

A `voice_rooms` row is persistent and discoverable while ACTIVE or LOCKED unless `is_discoverable`, visibility and community policies say otherwise. Each live broadcast creates a separate `voice_room_sessions` row with a new session ID. Ending, expiring or recovering a session does not expire or delete the room. Only explicit archive or suspension changes the durable room lifecycle. A room owner is independent from the HOST role for the current session.

15.3 Capacity and stage rules

- `max_speakers` is stage capacity (1–30); `max_participants` is total room capacity (2–1000). Configure total capacity greater than speaker capacity.
- The current session HOST counts toward the speaker limit and occupies a speaker seat. The persistent room owner is a separate role and does not automatically become HOST in every session. Listener count is total active participants minus stage occupants.
- Create numbered seat rows when a session is initialised. Seat number cannot exceed the configured room speaker limit.
- Admission and speaker promotion must serialize at session level or use equivalent concurrency-safe transactions. Concurrent joins/promotions must not overrun capacity.
- Only an active HOST or authorised MODERATOR can approve/decline requests or change speaker roles. Listeners cannot self-promote.
- Approving a request and assigning a seat must be one consistent operation. Demoting/leaving releases the seat; role transitions and moderation decisions are audited.
- Reject invalid transitions, including reopening terminal speaker requests. Enforce backend authorization in addition to database consistency triggers.

15.4 Room discovery, host recovery and lifecycle

Case	Target behaviour
Public room	Discoverable only when active/joinable and authorized for the viewer. Hide from discovery when inactive or ended.
Private room	Never shown in public search/discovery. Only authorised invitees/participants can discover existence and join.
Host leaves, listeners remain	Hide private room immediately. Start a short configurable hostless period (baseline proposal: 2 minutes); transfer to eligible host/co-host/moderator or end.
Host absent and room empty	Mark inactive, remove from discovery and start `grace_expires_at`; private default is 30 minutes, configurable.
Host returns during grace	Lock/recheck state; if not expired and authorization remains valid, resume same session and clear deadline.
Expiry	Worker rechecks under lock and ends/expires session; keep necessary audit/history metadata under retention policy.

Case	Target behaviour
Reconnect	Distinguish transient RECONNECTING/DISCONNECTED from explicit departure. Restore a valid provider connection when allowed; do not create duplicate active participant rows.

Provide Live Now and All Rooms discovery, filtering by community, topic/category and language, and show session live/offline state with participant and speaker counts. The default host/inactivity recovery grace window is 30 minutes. A backend worker reconciles heartbeats and RTC provider state, restores a temporarily disconnected session during the grace period or ends/expires the live session once recovery expires. It never removes the room. A separate short hostless handover window may be applied when other participants remain. Archived or suspended rooms leave ordinary discovery; locking prevents new admissions according to policy.

15.5 RTC adapter

Use a provider-neutral RTC interface such as `createSession`, `issueParticipantCredential`, `updatePublishPermission`, `removeParticipant`, `closeSession` and `reconcileProviderState`. Spring Boot is authoritative for roles and permissions; it issues short-lived provider credentials with listener/publisher permissions. PostgreSQL stores durable room/session records; Ably handles presence, live comments and control events; the configured RTC/SFU provider carries audio. Evaluate LiveKit and Cloudflare Realtime SFU before production selection. Expo/React Native is the client, not the audio transport.

15.6 Session-scoped ephemeral live comments

Voice comments are session-scoped temporary text messages. Insertion requires a participant in an ACTIVE session; comments remain recoverable during the INACTIVE grace state. PostgreSQL is the source of truth and the transactional outbox publishes comment-created events after commit.

When a session reaches ENDED or EXPIRED, delete its comment rows, discard queued undelivered comment-created outbox events, publish `VOICE_ROOM_COMMENTS_CLEARED`, and require clients to clear in-memory comments and ignore late created events for that session. A new session always begins with empty comments. The default recovery grace period is 30 minutes; a background worker must reconcile provider state and terminally expire abandoned sessions without expiring the persistent room.

Keep voice moderation actions and audit history separate from ephemeral comments. The clear event does not mean old comments remain queryable: after terminal cleanup, the comment rows are deleted. Test this against concurrent comment writes and session closure; reject new comments once the session is not ACTIVE.

16. API design and endpoint catalogue

The following is a target REST contract catalogue. Paths are proposals for implementation, not a claim that controllers already exist. Use a consistent `/api/v1` prefix, JSON envelopes, validation errors, pagination cursors, correlation IDs, idempotency keys for retry-sensitive creates, and server-derived actor identity.

Area	Representative endpoints	Rules
Identity / profile	`GET/PATCH /api/v1/me`; `GET/PATCH /api/v1/me/settings`; `POST /api/v1/privacy-requests`	OIDC identity mapped to profile; private fields require own-profile authorization.
Community / taxonomy	`GET /countries`; `GET /communities`; `GET /categories?module=FEED`; `POST /admin/categories`	Check category active state, country and module mapping.

Area	Representative endpoints	Rules
Feed	GET /api/v1/feed?mode=FOR_YOU|LATEST&cursor=...; POST /api/v1/posts; GET/PATCH/DELETE /posts/{id}; comments/reactions/saves/polls; POST /api/v1/content-views/batch; featured-item impression event	Filter visibility/moderation first; For You uses unseen-first ordering; Latest remains chronological; view events require meaningful exposure; featured cards obey scheduling, relevance and paid-frequency caps.
People Connect	`GET /people/discover`; follow/unfollow; view mutual connections; block/unblock	Respect privacy and symmetric block effects.
Direct messages	`GET /api/v1/conversations`; `POST /api/v1/conversations`; `POST /api/v1/message-requests`; `POST /api/v1/conversations/{id}/messages`; read/archive/mute operations	Context-specific uniqueness; initial message is tied to one pending GENERAL request; active Business Account membership, privacy, blocks and context permissions are checked for every operation.
Community chat	`GET /chat/rooms/{id}/messages`; `POST /chat/rooms/{id}/messages`; `POST /chat/rooms/{id}/read`	Active membership and chat moderation checks; outbox event after commit.
Voice rooms	`POST /voice-rooms`; `GET /voice-rooms/discover`; session join/leave/end; speaker workflow; `GET/POST /voice-sessions/{id}/comments`	Capacity and privacy rules; short-lived RTC credentials; comments are session-scoped, live-delivered and cleared when session ends/expires.
Business	`GET/POST /businesses`; locations/hours/reviews/saves; verification admin routes	Owner and moderator controls; address visibility; verification state.
Events	`GET/POST /events`; RSVP/save/report; calendar export	Mode-specific URL/address rules; timezone and capacity enforcement.
Jobs	`GET/POST /jobs`; categories/saves; internal apply/withdraw; report; admin review	Separate listing source, application method, employment type and category.
Spare Bag	`GET/POST /spare-bag/listings`; saves; contact requests; accept/decline/cancel/close/report	Route/date eligibility, listing visibility and sensitive handover details.
Matchmaking	GET /matchmaking/discover; POST /matchmaking/swipes; DELETE /matchmaking/likes/{targetUserId}; POST /matchmaking/rewind; POST /matchmaking/matches/{matchId}/unmatch; match list/report/block	Every route checks adult eligibility, profile status, mutual compatibility, blocks and visibility.

Area	Representative endpoints	Rules
Realtime/media	`POST /realtime/ably-token`; upload grant/complete; authorised media access	No API signing keys in clients; server-controlled capabilities and short-lived media access.
Admin / moderation	`GET /admin/reports`; report action; content hide/restore; role change; audit log	Least privilege, reason capture, access logging and audit.
Service marketplace	`GET /api/v1/service/categories`; provider profile/services; `GET/POST /api/v1/service/requests`; request responses/quotes; lead access; reviews/reports	Customer posts governed by SERVICE_JOB_POST. Contact unlock/chat governed by SERVICE_LEAD_ACCESS, idempotent per provider/request. No booking, checkout or service payment.

16.1 Common API requirements

- Use stable machine-readable error codes; do not reveal whether an inaccessible private object exists where that would leak sensitive information.
- Validate request sizes, pagination limits, enums, identifiers, URLs and upload state. Use server-side URL allow/deny checks for redirects and SSRF-sensitive fetch workflows.
- Idempotency is required for create operations that can be retried after timeouts, especially messages, mutual matches, contact requests, uploads and payments if introduced later.
- Use explicit API DTOs/allowlists. Do not serialize database entities directly; omit protected fields by construction.
- Use consistent UTC timestamps (`timestamptz`) and explicit IANA timezones for user-facing scheduled events.

17. Ably realtime contract

17.1 Channel conventions

Channel pattern	Purpose	Authorization
`community:{id}:feed`	Optional new-post/feed events	Subscriber must have current eligibility for the selected community/country scope.
`community:{id}:chat:general`	General Chat message events	Active membership and chat access required.
`conversation:{id}`	Direct message and conversation events	Only current authorized participants. Verify every token capability issuance.
`user:{id}:notifications`	User-specific notification events	Authenticated identity must exactly match the user represented by the channel.
`voice-session:{id}`	Room/session state, speaker request decisions	Only authorized participants/invitees; do not expose private room existence broadly.
`user:{id}:ephemeral`	Optional typing/activity hints	Short-lived and non-authoritative; not message history or a presence database.
`voice-room:{roomId}:session:{sessionId}`	Voice-room state and temporary live comment events, including	Only participants authorised for that specific active/grace session;

Channel pattern	Purpose	Authorization
	comment-created and comments-cleared	the backend issues capabilities and clients clear memory on the terminal clear event.

17.2 Authentication and publishing

- Client calls Spring Boot to obtain short-lived Ably token authentication. The backend derives the Ably identity from the validated OIDC principal.
- Never embed Ably API keys or signing secrets in Expo builds, Next.js public bundles or browser storage.
- Durable create/update/delete events are published by a transactional outbox worker only after database commit. Do not permit untrusted client publishing to durable message channels unless equivalent persistence and validation are implemented.
- Client-to-Ably publishing is appropriate for explicitly ephemeral events such as typing only when separately scoped, rate-limited and policy-checked.
- Capabilities must be narrowly scoped to the user's community, conversations, voice session and own notification channel. Avoid broad wildcards.

17.3 Event envelope

```
{
  "eventId": "uuid",
  "eventType": "chat.message.created",
  "schemaVersion": 1,
  "occurredAt": "2026-10-18T12:00:00Z",
  "aggregateType": "chat_message",
  "aggregateId": "uuid",
  "communityId": "uuid-or-null",
  "payload": { "messageId": "uuid", "roomId": "uuid", "sequenceNo": 123, "sender": {}, "body": "..." }
}
```

Consumers must tolerate duplicate, delayed and out-of-order events. `eventId` is the deduplication key. Use the sequence number and REST resource state for ordering and recovery; an Ably event is a delivery hint, not a substitute for authorized reads.

17.4 Capacity planning

The initial architecture target is a load test at 10,000 concurrently connected application clients. Do not treat provider headline limits as a capacity guarantee: benchmark simultaneous connections, channel subscriptions, publish rate, outgoing fan-out, reconnect storms, token renewal, outbox lag and message payload size. Verify live plan limits/prices before procurement because provider plans change.

18. PostgreSQL logical model and schema catalogue

The v10.10 strict clean-install SQL consolidates the v10.8 greenfield baseline with a Follow-derived mutual-connection view, persistent voice-room definitions and separately expiring sessions, scoped chat restrictions, poll lifecycle enforcement, backend-controlled featured placements, Business Account permission/audit/lifecycle support, push-delivery records, and Spare Bag country derivation. Static source inventory counts are not proof of successful database execution.

18.1 Domain table inventory

Domain	Representative tables (base and later additions)
Identity / community	countries; profiles; addresses; external_identities; communities; community_memberships; user_settings; user_privacy_settings; privacy_data_requests; global_user_roles; policy_documents; policy_acceptances
Country taxonomy / discovery	country_categories; country_category_modules; interests; user_interests; languages; user_languages; user_follows; user_mutual_connections (derived view); user_blocks; user_mutes
Media	media_objects; profile_media
Feed	feed_posts; feed_post_media; feed_post_comments; feed_post_reactions; post_saves; post_shares; feed_polls; feed_poll_options; feed_poll_votes; user_content_views; featured_items; featured_item_impressions
Chat / direct messaging	chat_rooms; chat_messages; chat_room_user_state; chat_message_mentions; chat_message_media; direct_conversations (context and personal/business sides); direct_conversation_participants; direct_messages; direct_message_media; message_requests; room/community chat restrictions in user_mutes
Voice rooms	voice_room_categories; persistent voice_rooms; voice_room_sessions; voice_room_participants; voice_room_seats; voice_room_speaker_requests; voice_room_invites; voice_room_moderation_actions; voice_room_messages (ephemeral by session)
Business Accounts / Directory	business_accounts; business_account_members; business_role_permissions; business_account_audit_events; business_categories; business_listings; business_locations; business_hours; business_media; business_reviews; business_saves
Events	event_categories; events; event_rsvps; event_saves; event_media
Jobs	job_categories; job_listings; job_listing_categories; job_media; job_saves; job_applications
Property / vehicle listings	property_types; country_property_types; property_listings; property_listing_communities; property_listing_media; property_amenities; property_listing_amenities; property_listing_saves; property_listing_contacts; vehicle_categories; vehicle_makes; vehicle_models; vehicle_variants; vehicle_listings; vehicle_listing_communities; vehicle_listing_media; vehicle_listing_features;

Domain	Representative tables (base and later additions)
	vehicle_listing_saves; vehicle_listing_contacts. Both listing tables store validated category-specific additional_attributes as JSONB.
Spare Bag	airports; spare_bag_categories; spare_bag_listings; spare_bag_listing_categories; spare_bag_listing_media; spare_bag_listing_saves; spare_bag_contact_requests
Matchmaking	matchmaking_genders; matchmaking_religions; matchmaking_profiles; matchmaking_preferences; matchmaking_preference_genders; matchmaking_preference_religions; matchmaking_preference_languages; matchmaking_swipes; matchmaking_matches, matchmaking_swipe_history
Notifications / trust / operations	notifications; notification_preferences; device_tokens; notification_deliveries; reports; moderation_actions; audit_logs; feature_flags; feature_access_rules; featured_placement_rules; app_settings; idempotency_keys; outbox_events; media_processing_jobs; analytics_events
Service marketplace	service_categories; service_category_translations; service_providers; service_provider_services; service_requests; service_request_media; service_request_responses; service_lead_access; service_reviews; service_reports; service_moderation_actions

18.2 Key constraints and indexes

- Unique community origin/destination pair, active community membership and category country/module checks.
- One General Chat per community via a partial unique index; direct conversations have a canonical ordered participant pair.
- Monotonic message sequence allocation for General Chat and direct conversations; read cursors cannot move backwards or exceed assigned sequence.
- Unique active voice participant per session/user; one active host per session; unique seat number per session and no seat beyond the speaker limit.
- One canonical matchmaking match per unordered user pair; one directional swipe decision per actor/target pair; mutual LIKE creates or activates the match.
- Job categories are separate, and published listings require an accountable actor, primary category and valid source/application contract.
- Foreign keys define lifecycle and delete behaviour; polymorphic report target IDs require service-layer existence/type checks.
- User/content view uniqueness is enforced by `(user_id, content_type, content_id)`; timestamps must be monotonic under delayed/retried view events.
- `featured_items` schedules a typed polymorphic content target. Dynamic target existence/type, audience scope, publication state and moderation are service-layer obligations because content_id cannot have a normal FK to multiple domain tables.

- Featured impression events use an idempotency key and require qualified visibility; retention/partitioning is volume-dependent.

18.3 Critical database versus application responsibilities

Database can enforce	Backend/service must still enforce
Foreign keys, uniqueness, enum/check constraints, seat/participant relations, message sequence assignment, some state transitions and integrity triggers.	OIDC identity validation, object-level access, membership policy, content DTO filtering, full job source verification, eligibility process, address visibility, matching policy, scoped Ably capabilities, signed media access and safe moderation UX.
Transactional row locks/unique indexes for contested state transitions when called through the intended transaction path.	Call the operation through a transaction with retries and safe idempotency; prevent alternate APIs from bypassing service invariants; reconcile provider state and async operations.

18.4 Clean-install and migration policy

- Do not apply the v10.10 strict clean-install SQL on top of an existing database. It is a greenfield clean-install baseline, not an in-place migration.
- Run the v10.10 strict clean-install SQL on a disposable PostgreSQL 15+ instance with PostGIS available; fix all syntax, type-resolution, trigger-order and semantic errors before production.
- Create versioned Flyway migrations for an existing deployment only after identifying its actual schema/version and data semantics.
- Run backfill/validation scripts, take and test backups, rehearse migration and rollback/forward-fix plans, and measure lock/downtime impact.
- Use least-privilege roles, private networking, connection pooling, statement timeouts, database health monitoring and restore rehearsals.

19. Media and Cloudflare R2

19.1 Media lifecycle

1. Client requests an upload grant for an allowed media purpose and object size/type.
2. Spring Boot checks the user, target object, quotas and allowed media scope before issuing short-lived upload instructions.
3. Client uploads bytes directly to R2 under the constrained key/operation, then calls the completion endpoint.
4. Backend/worker verifies file metadata and expected MIME/signature, runs required processing and marks the media READY or REJECTED.
5. Create a relationship from the business entity to `media_objects` only when ownership, access scope and processing status are valid.
6. Authorized media reads use signed URLs or backend-gated access with expiry. Revoke/delete operations must also remove or invalidate cached URLs when possible.

19.2 Security and processing

- Never assume a client-supplied MIME type or extension proves a file is safe. Enforce size limits, content type/signature checks and malware/safety policy.
- Strip EXIF/GPS metadata from public image renditions where appropriate; create thumbnails and image variants through the processing pipeline.
- Keep source and processed objects distinguishable, with lifecycle cleanup for abandoned uploads and orphaned media.
- Matchmaking hidden/blurred photos require separately generated, separately referenced renditions. Never hand out original object URLs for a hidden/blurred profile.

- Resume and application attachments, private addresses and other user-restricted media must use owner/authorized-viewer scopes, not public object access.

19.3 Video

For user-generated video, decide between worker-based FFmpeg processing and a managed video service before scale. HLS rendition creation, transcode queue, thumbnail generation, object retention, playback authorization and egress costs must be measured. Video provider choice is an operational decision, not required to change the core R2 metadata model.

20. Authentication, security and privacy

20.1 Identity and sessions

- Use one OIDC provider at runtime: Keycloak is the recommended baseline; Authentik is an alternative. Do not operate both as equal identity sources initially.
- Expo uses Authorization Code + PKCE through the system browser and operating-system-secured token storage.
- Next.js should prefer an HTTP-only, Secure, SameSite session cookie/BFF approach; avoid long-lived access tokens in localStorage.
- Spring Security validates issuer, signature/JWKS, expiration, configured audience and required claims/scopes. Map stable OIDC `(issuer, subject)` to an internal profile.
- Use MFA/step-up authentication for privileged administrators where feasible; implement refresh/revocation, account recovery and secret rotation.

20.2 Sensitive matching and location data

- Adult eligibility is enforced on the server for every matchmaking endpoint and deep link. Date-of-birth confirmation timestamp alone is not age proof.
- Exact address/coordinates are used only for the purpose and audience authorized by policy. Return distance bands or approximations, not exact coordinates.
- Name/photo visibility must be enforced in query projections and DTOs. Never include hidden originals in JSON, error objects, logs, analytics or Ably events.
- Religion, children, marital status, lifestyle and other matching preferences are sensitive personal attributes. Restrict operational access, establish retention policy and do not expose internal scoring or private preferences.
- Block applies reciprocal exclusion from discovery/contact. Reporting should not reveal who reported a user to that user.
- Country/community visibility and membership checks apply to search, notifications, bookmarks and media access, not merely primary list endpoints.

20.3 Content safety and URL policy

- Validate HTTP(S) for external event links, external application URLs; do not permit `javascript:`, `data:` or unsafe redirects.
- Implement reporting, rate limits, spam controls, moderation queues, account restrictions, appeals where offered and immutable audit history.
- Provide policy version acceptance records for relevant community rules, privacy terms and Spare Bag conditions.
- Define retention windows for messages, reports, deleted profiles, resumes, private media and audit logs. Build privacy export/deletion workflows that respect lawful audit-retention obligations.

21. Background jobs, notifications and reliability

Worker / recurring operation	Responsibility / invariant
Transactional outbox publisher	Claim rows safely, publish to Ably/push/internal destination, retry with backoff, mark delivered and alert on repeated failure; never publish uncommitted durable state.
Job expiry	Close/expire published jobs after `closes_at`; stop discovery and preserve application/history records.
Spare Bag expiry	Expire listings and contact requests after travel windows or deadlines; hide from discovery and preserve necessary audit information.
Voice lifecycle worker	Recheck grace/hostless deadlines under lock, close expired sessions, clear discovery visibility, reconcile provider state and release stale participants/seats.
Notification delivery	Send user-specific notification through Ably for active clients and APNs/FCM/Expo push when appropriate; honor preference/privacy and avoid sensitive preview text.
Media processing	Verify objects, create variants/transcodes, update status, retry transient failures and clean orphaned uploads.
Privacy requests	Process export/deletion/rectification requests idempotently, record failures and audit completion status.
Search/index repair	Repair/rebuild search vectors or derived aggregates if updates were missed; monitor lag and errors.

21.1 Failure behaviour

- If Ably is unavailable, accepted durable messages still persist and remain retrievable by REST; outbox events retry when the provider recovers.
- If R2 upload completion fails, the object remains non-public until validated and attached; cleanup removes abandoned uploads after a safe grace period.
- If RTC provider join fails, the API reports a retryable/terminal reason and does not leave a phantom active participant. A reconciliation worker repairs drift.
- Workers are idempotent and safe under duplicate scheduling. State is determined by database timestamps and persisted transitions, not mobile timers.
- Use bounded retries, exponential backoff, poison-message handling, dead-letter/manual replay procedures and alert thresholds for growing outbox backlog.

22. Deployment, observability and scaling

22.1 Environments

Environment	Controls
Local development	Docker Compose or equivalent for PostgreSQL, OIDC provider, Spring Boot, Next.js and worker; mocked/test Ably, R2 and RTC credentials; synthetic seed data.
Staging	Isolated database and identity realm, R2 bucket, Ably app/key and RTC project. Apply schema from clean state, practice migrations/restores, run real mobile builds and load/security tests.

Environment	Controls
Production	TLS, private database network, separate secrets/buckets/projects, backups/PITR where available, resource limits, monitoring/alerts, access review, release approval and tested rollback/forward-fix procedure.

22.2 Deployment topology

Component	Deployment notes
Spring Boot API	Stateless application nodes behind TLS reverse proxy/load balancer; health/readiness probes; bounded DB pool; structured logs and trace/correlation IDs.
Worker / scheduler	May initially run beside backend or as a separate process; use safe DB claiming/locking so multiple instances do not duplicate non-idempotent work.
PostgreSQL	Private network, persistent storage, monitored disk/connection/replication state as applicable; automated backups and restore drills.
Next.js	Vercel or managed hosting; secure web session/BFF setup and server-only secrets; no private database credentials or Ably signing key in public bundles.
R2 / Ably / RTC	Separate production credentials, scoped permissions, usage alarms, secret rotation and provider outage procedures.
Observability	Metrics, central structured logs, tracing, uptime checks, alert routes and documented incident runbooks.

22.3 Initial service-level targets (targets, not guarantees)

Signal	Initial target / measurement
Normal REST API	p95 below 400 ms for typical non-search reads/writes at normal load; exclude upload and RTC setup.
First feed content	Useful first page in under 2 seconds on a supported mobile device/network; measure client-side.
Realtime delivery	p95 under 1 second from commit to client receipt under normal regional conditions; include outbox lag.
Voice join	Authorized join generally under 5 seconds on a healthy network/provider; instrument permission and provider failures.
Concurrency	Load test 10,000 concurrently connected Ably clients with realistic subscriptions, publish rate, fan-out and reconnect storms.
Recovery	Choose explicit RPO/RTO; measure backup freshness and demonstrate restoration to a clean host before production.

22.4 Monitor

- API latency/error rate, auth failures, rate limits, failed URL validation and object-level authorization denials.
- PostgreSQL connection pool saturation, slow queries, lock waits, disk growth, backup/restore results and migration duration.
- Outbox age/backlog, Ably token renewal failures, event delivery errors, push failures and notification latency.
- Voice join failures, active participant drift, seat/capacity rejections, hostless timeouts, provider errors and session reconciliation.
- Matchmaking discovery volume, swipe rates, match creation idempotency conflicts, adult-gate denials, report/block actions and privacy-sensitive DTO audit checks.
- external links failing validation/checks, job/Spare Bag expiry backlog and media processing queue age.

23. Testing and acceptance criteria

Release is blocked until schema installation, security, privacy, concurrency and module end-to-end tests pass in staging. Static SQL checks are not a substitute for execution on the selected PostgreSQL version.

23.1 Database install and migration

- Execute the complete v10.10 strict clean-install SQL on a clean PostgreSQL 15+ disposable database with PostGIS enabled; test transaction completion and confirm tables, enums, indexes, functions and triggers in system catalogs.
- Exercise foreign-key deletes/update behaviours for addresses, profiles, community lifecycle, business locations and every ON DELETE contract.
- Run migration rehearsal from the exact deployed schema version only after a separate migration is authored; verify data backfills, lock duration, rollback/forward-fix and backup restoration.
- Run a static schema diff and migration test in CI before merging database changes.

23.2 Chat and realtime

- Insert community and direct messages concurrently; prove sequences are assigned uniquely and monotonically within each room/conversation.
- Reject backwards read-cursor updates and cursor values beyond the allocated sequence; prove unread counts exclude the reader’s own messages according to product policy.
- Latest direct-message preview reflects latest eligible message after insert/edit/soft-delete; edits to older messages do not incorrectly replace a newer preview.
- Outbox event is committed atomically with durable message; duplicate or reordered Ably delivery does not duplicate UI messages; REST recovers disconnect history.
- Non-members and non-participants cannot read, publish, mark read, access media or subscribe to private channels. Test token renewal and revoked access.
- Test separate threads for the same participant pair on two different listings/jobs; repeat opens of one context must reuse its thread. Test GENERAL request/accept/decline, one initial message only while REQUESTED, privacy options including MUTUAL_FOLLOWERS, block precedence and all user/business endpoint combinations.

23.3 Voice rooms

- Concurrently fill all speaker seats and remaining listener slots; no transaction can exceed `max_speakers` or `max_participants`.
- Confirm the host occupies one stage seat; test host transfer, moderator permission, approval/decline, duplicate request, self-promotion rejection, demotion and seat release.
- Terminal speaker requests cannot be reopened. User cannot approve their own request unless an explicit product rule says otherwise; active host/moderator status is checked at action time.
- Test leave/rejoin history, temporary reconnect, duplicate join, participant expiry and provider/session reconciliation.

- Test private-room visibility, invitation checks, hostless timeout, empty-room grace period, host return before deadline and race between expiry worker and rejoin.
- Test live voice comments are broadcast via the outbox; INACTIVE/grace preserves them; ENDED/EXPIRED removes rows and pending comment events and emits a clear event; late writes after terminal status fail.

23.4 Matchmaking and user safety

- Deny under-18, missing-DOB, deleted, suspended, inactive or unconfirmed/ineligible users at every matchmaking route and media-access path.
- Test mutual-preference matching with age, gender, distance, verified-only, children, religion and language combinations; verify missing attribute semantics and that preferences apply in both directions.
- Test LIKE/PASS behaviour and simultaneous reciprocal likes. One reciprocal LIKE pair must yield exactly one match and one accepted conversation.
- Exercise name/photo VISIBLE, BLURRED and HIDDEN modes. Ensure original photo media IDs/URLs and exact address/coordinates never leak through list/detail/search/recommendation/notification/Ably/logging outputs.
- Block in either direction; existing match/chat becomes restricted according to policy; neither profile appears to the other and no further messages/likes/notifications leak.
- Test report submission, attachment of relevant evidence, reviewer access, moderation action and reporter confidentiality.

23.5 Jobs, addresses and taxonomy

- Job coverage includes employer-direct warehouse work, recruiter submissions, community-shared posts, community organisations and aggregated sources. Source type remains independent of application route.
- Published jobs require valid accountable user, primary job category and valid HTTP(S) URL where external application is used; internal application table rejects external-only jobs.
- Address permission checks prevent private matchmaking locations and Spare Bag handover points appearing on public pages. Public business branch addresses can show authorised fields.
- Feature-specific categories remain separate; category country/module/scope and country-wide discovery filters are enforced consistently.

23.6 Media, accessibility and operations

- Test signed upload expiry, MIME/signature mismatch, oversized upload, orphan cleanup, EXIF removal, blurred rendition ACLs and private attachment access.
- Test screen reader labels, text scaling, keyboard navigation on web, denied microphone permission, background/foreground app transitions and offline retry states.
- Load test feed pagination, voice discovery, 10,000 realtime clients, message fan-out, reconnect burst, DB pool limits, search and worker contention.
- Demonstrate restore to a fresh host, secret rotation, outbox replay, feature disablement and a documented incident/rollback or forward-fix process.
- Feed discovery / promotions: unseen-first ordering is stable across cursor pages; a fetched or off-screen card is not marked viewed; view upserts are idempotent and monotonic; removed/ineligible targets disappear even when featured; paid placements are labelled and frequency-capped; impressions deduplicate on retry and obey retention policy.

23.7 Service marketplace

- Test personal and Business Account provider ownership exclusivity, accepted business-member authorisation, category translation, service request lifecycle, media access, radius edits and country/category filtering. Test ST_DWithin distance matching and ST_Distance sorting with known geography fixtures and inspect representative query plans.
- Test SERVICE_JOB_POST and SERVICE_LEAD_ACCESS using the shared billing engine; verify unconfigured/disabled entitlements fail closed, quantity_limit NULL means unlimited, zero denies usage and positive limits use the configured reset period. No price/quota is hard-coded by this schema update.

- Run two concurrent lead-access requests for the same provider/request and prove one access record and one billing usage record are created. Enable chat, then unlock contact details and prove no second usage unit is charged. Ensure exact address and contact data remain hidden until permitted access.
- Test service responses/quotes, customer notifications, reports, moderation audit, review eligibility after recorded interaction and rejection of self-reviews/provider self-responses. Confirm the user-facing text does not claim verified job completion and no booking/escrow/payment lifecycle is present.

24. Delivery roadmap

Phase	Deliverables	Exit gate
0. Design hardening	Confirm provider/environment details, privacy and retention, country-wide visibility policy, source-attribution rules, age-assurance approach and migration strategy.	Architecture decisions and threat model reviewed; staging plan approved.
1. Foundation	OIDC, profiles, countries/communities/memberships, roles, privacy settings, address model, interests/languages, policy acceptance and R2 media foundation.	Onboarding, authorisation, backup and taxonomy/address integrity tests pass.
2. Feed + General Chat	Feed, categories, media, comments/reactions/polls, chat room/message APIs, sequence read cursors, Ably token auth/outbox.	Durable-before-realtime, reconnect history, scope and read-cursor tests pass.
3. People Connect + DMs	Discoverability, follow/friend graph, blocks, message requests, direct conversations and read cursors.	Privacy, block, duplicate-message and nonparticipant access tests pass.
4. Voice rooms	One RTC provider, room/session lifecycle, fixed stage seats, high listener capacity, speaker requests, host controls, private invites and expiry.	Concurrency and provider integration, seat/capacity and timeout tests pass.
5. Directory + events	Shared address locations, business ownership/reviews, events/RSVP.	Business, address privacy and event mode tests pass.
6. Jobs + Spare Bag + matchmaking	Full job source/employment taxonomy, internal/external application flow, route/date contact workflow, adult-only profile matching and mutual chat.	Anti-scam, job integrity, adult gate, matching privacy and block/report checks pass.
7. Hardening and release	Universal search, language experience, accessibility, monitoring, load/security tests, migration rehearsal and store release preparation.	10k realtime test, backup restore, privacy/security sign-off and release gates pass.

25. Architecture decisions and open launch gates

25.1 Architecture decision records

ADR	Decision	Reason / implication
ADR-001	Spring Boot modular monolith	Keep transactions and domain invariants straightforward while defining boundaries for future extraction.

ADR	Decision	Reason / implication
ADR-002	Self-hosted PostgreSQL is canonical application store	Matches data-control goals; requires private networking, backups, restore tests and operational ownership.
ADR-003	External OIDC provider	Avoid custom password infrastructure; bind application profile via `(issuer, subject)`.
ADR-004	Ably for app realtime; PostgreSQL/outbox for durability	Avoid custom WebSocket operations and recover messages via REST after outages/reconnects.
ADR-005	Cloudflare R2 for object bytes	Object storage independent of relational records; processing and signed-access policy remain application responsibilities.
ADR-006	One permanent General Chat; temporary user-created voice rooms	Avoid permanent voice rooms and clarify who controls each temporary live session.
ADR-007	Destination-country practical taxonomy	Reuse local categories across origin communities in the same current country.
ADR-008	Voice-provider adapter, one provider per session	Avoid coupling application rules to provider details and avoid dual-provider operation for one session.
ADR-009	Profile-based matchmaking with mutual likes	Profiles/preferences drive discovery; reciprocal LIKE creates one match and enables normal chat; age gate and privacy rules are mandatory.
ADR-010	Shared address model plus module-specific taxonomies	Reduce duplicated address fields without forcing virtual entities or airport records into the same model; preserve appropriate module-specific classification.

25.2 Must be confirmed before production

- Choose the production voice provider for the first release (LiveKit Cloud/self-hosted LiveKit or Cloudflare Realtime SFU). Confirm RTC costs and operational ownership.
- Choose age assurance/verification standard for matchmaking; a stored confirmation timestamp is not proof on its own.
- Finalize matchmaking semantics for unknown/missing profile attributes, hard filters versus soft scoring, and when verification is considered sufficient.
- Choose exact public voice-room inactivity timeout and hostless grace period; v9 defaults are configurable proposals.
- Define address sharing for business locations, event venues, workplace addresses and private Spare Bag handover points.
- Confirm external aggregated job-listing permission/attribution and moderation standards.
- Set retention durations for messages, reports, sensitive matchmaking data, resumes/media, audit logs and deleted account records.
- Choose RPO/RTO and test restore/backups; establish incident response and provider outage playbooks.

- Before any launch, execute schema SQL in staging, repair runtime findings and prepare proper versioned migrations from the actually deployed database.

Appendix A. Historical v10.8 schema inventory (superseded by Appendix K)

Historical reference only: this appendix reflects an earlier schema inventory and is superseded by the v10.10 SQL deliverables and Appendix K. Do not use this table as the current inventory. The current baseline contains the objects described by `Diasporum_PostgreSQL_Schema_v10_10.sql`; use that file for exact columns, constraints, indexes, functions and triggers.

Domain	Primary records
Identity / community	countries, profiles, addresses, external_identities, communities, community_memberships, user_settings, user_privacy_settings, privacy_data_requests, global_user_roles, policy_documents, policy_acceptances
Taxonomy / discovery	country_categories, country_category_modules, interests, user_interests, languages, user_languages, user_follows, friend_requests, friendships, user_blocks, user_mutes
Feed / media	media_objects, profile_media, feed_posts, feed_post_media, feed_post_comments, feed_post_reactions, post_saves, post_shares, feed_polls, feed_poll_options, feed_poll_votes, user_content_views, featured_items, featured_item_impressions
Community chat / direct messaging	chat_rooms, chat_messages, chat_room_user_state, chat_message_mentions, chat_message_media, context-aware direct_conversations, direct_conversation_participants, direct_messages, direct_message_media, message_requests
Voice rooms	voice_room_categories, voice_rooms, voice_room_sessions, voice_room_participants, voice_room_seats, voice_room_speaker_requests, voice_room_invites, voice_room_moderation_actions, session-ephemeral voice_room_messages
Directory / events	business_categories, business_listings, business_locations, business_hours, business_media, business_reviews, business_saves, event_categories, events, event_rsvps, event_saves, event_media
Jobs	job_categories, job_listings, job_listing_categories, job_media, job_saves, job_applications
Property / vehicle listings	property_types, country_property_types, property_listings, property_listing_communities, property_listing_media, property_amenities, property_listing_amenities, property_listing_saves, property_listing_contacts, vehicle_categories, vehicle_makes, vehicle_models, vehicle_variants, vehicle_listings, vehicle_listing_communities, vehicle_listing_media, vehicle_listing_features, vehicle_listing_saves, vehicle_listing_contacts; JSONB

Domain	Primary records
	additional_attributes plus dedicated core filter columns.
Spare Bag / matchmaking	airports, spare_bag_categories, spare_bag_listings, spare_bag_listing_categories, spare_bag_listing_media, spare_bag_listing_saves, spare_bag_contact_requests, matchmaking_genders, matchmaking_religions, matchmaking_profiles, matchmaking_preferences, matchmaking_preference_genders, matchmaking_preference_religions, matchmaking_preference_languages, matchmaking_swipes, matchmaking_matches, matchmaking_swipe_history
Notifications / operations	notifications, notification_preferences, device_tokens, reports, moderation_actions, audit_logs, feature_flags, feature_access_rules, app_settings, idempotency_keys, outbox_events, media_processing_jobs, analytics_events
Service marketplace	service_categories, service_category_translations, service_providers, service_provider_services, service_requests, service_request_media, service_request_responses, service_lead_access, service_reviews, service_reports, service_moderation_actions

Appendix B. Historical lifecycle and API conventions (superseded where inconsistent with Appendix K)

Area	Lifecycle / convention
Content	DRAFT → PENDING_REVIEW → PUBLISHED, with HIDDEN/REMOVED/REJECTED/EXPIRED as policy states. Only authorized published content appears to ordinary users.
Jobs	DRAFT → PENDING_REVIEW → PUBLISHED → PAUSED/CLOSED/EXPIRED. External application URL required only for EXTERNAL route; internal applications are stored only for INTERNAL jobs.
Matchmaking	Profile DRAFT/ACTIVE/PAUSED/SUSPENDED/DELETED; directional swipe LIKE/PASS; match MATCHED/CLOSED/BLOCKED; a mutual LIKE creates one match and accepted chat.
Voice room	Room and session have separate state; participants have role and connection state; explicit leave sets `left_at`; rejoin creates a new participation record.
Realtime	Ably events are at-least-once/possibly delayed; deduplicate by `eventId`, use sequence numbers for chat ordering and REST for durable recovery.

Area	Lifecycle / convention
Errors	Stable error codes, validation detail safe for clients, correlation ID; avoid revealing existence of private resources.
Time	Store instants as `timestamptz`/UTC; store an IANA timezone identifier for event presentation and recurrence rules.

Appendix C. Historical source artifacts and limits

The v9.0 base design was prepared using the existing Diasporum Full System Design v7.0, `Diasporum_PostgreSQL_Schema_v9.sql` and the v9.0 Change and Static Audit Report available in the working artifact set. The current v10.6 companion SQL is a greenfield clean-install candidate, not an in-place migration from any deployed version. The earlier v10.4 baseline remains unchanged.

- Historical v9.0 base-schema static audit counts: 107 table definitions, 134 indexes, 75 functions and 106 triggers; zero duplicate table/index/function/trigger names were reported by the static checker, and explicit foreign-key target table/column checks passed.
- The static checker does not execute SQL, compile PL/pgSQL in PostgreSQL, prove trigger ordering, or prove concurrency/security behavior. Runtime verification remains a staging requirement.
- The historical v9.0 file was a clean-install candidate, not a migration from v8.1. The current v10.6 companion SQL is also a clean-install candidate and is not an in-place migration from any deployed version.
- Pricing and plan limits for Ably, RTC, R2 and hosting are intentionally not guaranteed in this document; re-check provider details at procurement and before production.

Final implementation warning: Before production, execute the v10.6 SQL on a clean PostgreSQL staging instance; repair runtime findings; produce migrations from the actual deployed database if upgrading an existing deployment; configure grants, secrets, backups and monitoring; and pass the concurrency, security, privacy, accessibility, JSONB-validation and restore tests described in this document.

Appendix D. Historical v10.0 updates: marketplaces, visibility and billing

This section records the product decisions added after the v9.0 baseline. The companion SQL is Diasporum_PostgreSQL_Schema_v10.sql. V10 is a clean-install candidate based on v9.0, not an in-place migration. Pricing and quotas below are configurable examples, not final commercial commitments.

D.1 Property marketplace (historical)

Property listings support SALE and RENT with a full description, exact decimal price and currency, optional rent period, bedrooms, bathrooms, reception rooms, floor area and unit, furnished status, rental deposit, bills included, minimum tenancy, availability date, tenure/condition, public location, contact preferences, ordered media, amenities, saves, moderation and expiry. No online booking, checkout, escrow or transaction processing is included. In v10.5, additional category-specific attributes are stored in an object-shaped JSONB column and validated by the backend; high-value filter fields remain dedicated columns.

Property types use a canonical property_types catalogue mapped through country_property_types. Each country controls which types are enabled and the local display label. A property listing references the country/type pair so a type cannot be used outside its configured country.

D.2 Vehicle marketplace (historical)

Historical v10.0 note: vehicle listings supported SALE and RENT, with relational make/model catalogues and optional variants. The current v10.6 vehicle fields retain separate model_year and registration_year, plus drive_type, previous_owners and JSONB additional_attributes. No insurance_group is included.

D.3 Listing ownership and Business Accounts

Both personal users and Business Accounts can publish eligible property, vehicle, event and job listings. Each listing has one ownership context: personal owner or business account. created_by_user_id records the acting user. Public business profiles belong to a Business Account; one account can manage multiple profiles and locations under one account-level business subscription. The account supports owner, administrator, editor, listing-manager and billing-manager roles. Payment does not automatically verify a business.

D.4 Shared country-bounded visibility

The common visibility modes are COMMUNITY, SELECTED_COMMUNITIES and COUNTRY. COMMUNITY targets one community; SELECTED_COMMUNITIES targets an explicit list of communities; COUNTRY targets eligible users across the selected country. The country is the hard upper boundary: UK content must not be visible through US discovery. Selected communities must have the same destination country as the listing/profile. Backend authorization must enforce the same rule for search, direct links, saved items, recommendations, notifications and media access. Scope is separate from content category and physical location.

The visibility model applies to feed posts, events, jobs, business profiles/listings, property listings, vehicle listings and matchmaking profiles. Classification is independent from audience scope; audience scope never expands because of a category or paid promotion.

D.5 Configurable billing model

Billing is centralised and entitlement-driven. Plans grant ongoing feature permissions and quotas; one-time products grant a specific action/credit or time-limited benefit; usage records track quota consumption where required. Feature modules ask the central entitlement service whether an action is allowed and do not implement their own payment logic.

Illustrative personal plans: FREE, SILVER and GOLD. Illustrative business plans: BUSINESS_FREE, BUSINESS_SILVER and BUSINESS_GOLD. The business subscription belongs to the Business Account, not each public business profile. A personal account may manage a business account without its personal plan being replaced. A listing purchase can be purchased independently of a subscription if configured. Listing publication and featured promotion are distinct products.

The current v10.2 SQL seeds 16 billing_features, including BUSINESS_TEAM_SEATS and SPARE_BAG_LISTING_PUBLISH. The feature table remains the canonical registry; no business module should hard-code plan names or quota values. Plan access is configured in billing_plan_entitlements, one-off/time-limited benefits use billing_entitlement_grants, and measurable consumption is recorded in billing_usage_records. The two new features have disabled entitlement placeholders until administrators configure and approve their plan limits.

D.6 Billing lifecycle and controls

The schema includes plan and product catalogues/prices, subscriptions, orders/order items, payments, refunds, entitlement grants, usage records, idempotent webhook events and billing audit events. Subscription ownership is exactly one personal user or one business account. Payment-provider events must be idempotent; a benefit is not granted until the relevant payment is confirmed. Quota consumption must be concurrency-safe. Failed, refunded, expired or canceled purchases must follow explicit entitlement-revocation/grace policies. Payment never bypasses country/community scope, moderation, age eligibility or verification.

D.7 Suggested launch approach

Launch with a small number of plans and use configuration for quotas. Keep browsing and essential core community features free. Consider standard personal listings free at launch and monetise optional promotions first, unless validated demand supports per-listing fees. Business subscriptions should provide profile, location, team, analytics and promotional tools; one-time listing fees remain separate. Treat sample prices as hypotheses to test.

D.8 V10 static-review limits

The v10.2 SQL is a clean-install candidate derived from the v9.0 baseline and subsequent additive revisions; it has not been executed against a PostgreSQL server. Before launch, run it on a clean PostgreSQL staging instance, resolve

all SQL/constraint/function/trigger findings, validate selected-community country consistency under concurrent writes, verify all existing scope constraints, and run security, entitlement, refund, webhook idempotency, quota concurrency and restore tests. Prepare a migration from the actually deployed database separately; do not apply this clean-install candidate over an existing database.

Historical v10.2 principal schema records included business_accounts, business_account_members, property_types, country_property_types, property_listings and supporting media/amenity/contact/save/audience tables, vehicle_makes, vehicle_models, vehicle_variants, vehicle_categories, vehicle_features, vehicle_listings and supporting media/feature/contact/save/audience tables, business/event/job/matchmaking selected-community mappings, billing plans and entitlements. The current v10.5 table inventory is in Section 18 and Appendix G.

Appendix E. Historical v10.2 follow-up decisions

This v10.2 follow-up records the address-normalization decision, the complete list of 16 billing_features seeded in the companion SQL, default-disabled plan entitlement placeholders for the two newly added features, and evaluation of RevenueCat as a payment integration. RevenueCat remains a candidate rather than the selected provider.

E.1 Address normalization (Option A)

Property listings, vehicle listings, events and job listings no longer duplicate city/region/postcode values when they reference a shared addresses row. Use address_id (or workplace_address_id for jobs) as the single location reference, and obtain city, region, postal_code, country and coordinates by joining addresses. When only an approximate city is known, create an addresses row containing the country and city while leaving street/coordinate fields null; this preserves one location source without requiring a precise street address.

Linked addresses now use ON DELETE RESTRICT for property, vehicle, event and workplace/job references. This avoids silently detaching a listing from its location. Address visibility and entity authorization remain mandatory: the existence of an address UUID never grants permission to reveal a private address. The address country must match the listing/event/job country; retain or strengthen the existing service/trigger validation for that invariant.

Indexes support country/city lookup through addresses and address-to-listing joins. Business listings retain their top-level city field intentionally because a business profile can represent a service/base area while its business_locations table holds multiple branch addresses; it is not a one-to-one duplicate of a single address. Profile city and airport city are also separate domain attributes, not duplicate listing address columns.

E.2 Current billing feature catalogue (v10.6)

The following table lists the 16 feature keys seeded by the current v10.6 SQL, with purpose, kind and reset behaviour. This is the feature catalogue, not a claim that every feature is enabled for every plan. Administrators configure each plan entitlement and quota before launch. For an enabled entitlement, quantity_limit = NULL means unlimited; a finite limit requires a configured reset_period. DIRECT_MESSAGE_SEND uses a separate quota for each sender-recipient pair and the reset period is selected per plan.

Feature key	Display name	Purpose	Kind	Reset period
POST_CREATE	Create posts	Publish feed posts	QUOTA	MONTH
COMMENT_CREATE	Create comments	Comment and reply	QUOTA	MONTH
DIRECT_MESSAGE_SEND	Send messages	Count each accepted outgoing message per recipient; quota and reset period are set per plan	QUOTA	Per-plan configurable
MATCHMAKING_INTERACT	Matchmaking interaction	Like/pass and other permitted matchmaking actions	QUOTA	MONTH
VOICE_ROOM_JOIN	Join voice rooms	Join eligible public voice rooms	LIMIT	DAY
VOICE_ROOM_HOST	Host voice rooms	Create/host persistent voice rooms	QUOTA	MONTH
EVENT_CREATE	Create events	Publish eligible events	QUOTA	YEAR
PROPERTY_LISTING_PUBLISH	Publish property	Publish one eligible property listing	QUOTA	YEAR
VEHICLE_LISTING_PUBLISH	Publish vehicle	Publish one eligible vehicle listing	QUOTA	YEAR
JOB_LISTING_PUBLISH	Publish job	Publish one eligible job listing	QUOTA	YEAR
BUSINESS_PROFILE_CREATE	Create business profile	Maximum number of active public business profiles owned by a Business Account	LIMIT	NEVER (concurrent cap)
BUSINESS_LOCATION_CREATE	Create business location	Maximum number of active business locations owned by a Business Account	LIMIT	NEVER (concurrent cap)
LISTING_PROMOTE	Promote listing	Buy/activate featured placement	QUOTA	MONTH
BUSINESS_ANALYTICS	Business analytics	Access business analytics	BOOLEAN	—
BUSINESS_TEAM_SEATS	Business team seats	Maximum number of active members in a Business Account allowed by its plan. Enforce when invitations are accepted or members are activated/changed.	LIMIT	NEVER (concurrent cap)
SPARE_BAG_LISTING_PUBLISH	Publish Spare Bag listing	Allowance for publishing eligible Spare Bag sender requests or passenger offers. Configure quantity_limit and reset_period per plan before enabling.	QUOTA	Per-plan; configure before enabling
SERVICE_JOB_POST	Post service job	Publish service requests; price/quota configured through the shared entitlement system	QUOTA	Per-plan configurable
SERVICE_LEAD_ACCESS	Access service lead	Unlock permitted contact details or in-app chat once per provider/request	QUOTA	Per-plan configurable
SERVICE_QUOTE_SEND	Send service quote	Reserved optional feature; configure and enable only if quote submissions are later made billable	QUOTA	Per-plan configurable

Feature access and quotas are configured in billing_plan_entitlements. Purchase, trial, promotion and admin grants use billing_entitlement_grants; metered usage uses billing_usage_records. In v10.6, disabled placeholders remain for BUSINESS_TEAM_SEATS on business plans and SPARE_BAG_LISTING_PUBLISH on personal plans. Configure a numeric limit for seats; configure quantity_limit and reset_period for Spare Bag publication; then explicitly enable each entitlement. Enforce membership caps and usage quotas transactionally in the backend. Payment never bypasses moderation, verification or audience rules.

E.2.1 Configuration of plan-specific billing features

BUSINESS_TEAM_SEATS controls the maximum number of active Business Account members, a concurrent cap rather than a resetting quota. SPARE_BAG_LISTING_PUBLISH controls publication allowances for eligible personal-user-owned sender requests and passenger offers. Its quota reset period is deliberately left unconfigured so it can be set per plan. Both feature definitions are seeded, but their plan entitlements start disabled; approve the launch policy, set limits and reset periods, and then enable each entitlement.

Deferred ideas, not current catalogue entries: SAVE_CONTENT (saving/bookmarking feed posts, property listings, vehicle listings and other supported content for normal/personal users), AD_FREE_EXPERIENCE (BOOLEAN), MATCHMAKING_BOOST (QUOTA), and MEDIA_STORAGE_LIMIT (LIMIT). SAVE_CONTENT is an explicitly recorded future requirement but is deliberately absent from v10.6. Add deferred items only after the relevant monetization and product rules are approved. If Spare Bag publication is later sold as a one-off billing product, extend billing_products.target_type to support SPARE_BAG in that future revision; the current feature quota is a subscription entitlement, not a one-off purchase.

E.3 RevenueCat evaluation (not selected yet)

RevenueCat is a candidate for simplifying Apple App Store and Google Play in-app subscriptions and purchase lifecycle handling for the React Native/Expo mobile app. Its React Native/Expo SDK requires a development build rather than relying only on Expo Go. RevenueCat also documents web purchase support through RevenueCat Billing (Stripe-backed) or supported web billing integrations; web products are configured separately from native store products, while entitlements can be coordinated using a stable shared App User ID. Confirm that the exact current SDK/version, Expo SDK and target build pipeline are supported before implementation.

Proposed integration boundary: the Diasporum Spring Boot backend remains the authorization and entitlement enforcement point. The mobile/web clients use RevenueCat SDKs for purchase flows, but client-reported entitlement state alone does not grant server access. Configure authenticated (and, where enabled, HMAC-verified) webhooks; persist each event in billing_webhook_events before asynchronous processing; deduplicate by provider_key plus event ID; map the external store product to a local plan or one-off product through billing_provider_products; update billing_subscriptions/payments and the local entitlement records; and retry safely. Use the internal profile UUID as the stable RevenueCat App User ID after login, with an explicit anonymous-to-authenticated identity/alias policy. Never use email as the subscription identity key.

Schema support: billing_provider_products supports distinct product identifiers for different stores mapping to the same plan, or store-specific one-off products mapping to billing_products. Existing provider_key/provider_customer_id/provider_subscription_id fields and billing_webhook_events remain provider-neutral. For RevenueCat, agree a mapping convention for App User ID, product/store identifiers, transaction IDs, environment, cancellation/expiration and event IDs before coding; retain the original webhook payload as audit evidence without logging secrets.

Decision gates before selection: compare total cost at expected monthly tracked revenue and transaction volume; check current webhook plan availability and delivery/retry behaviour; test renewals, cancellations, billing issues, grace periods, refunds, family/alias/transfer events, restore purchases, sandbox vs production, customer identity merges, one-time consumables and business-account subscriptions; confirm App Store/Google Play policy fit and web checkout/payment options; and test provider outage/replay recovery. RevenueCat pricing and features can change and must be re-checked before procurement. Do not make RevenueCat a hard schema dependency until this evaluation is accepted.

Official references checked on 19 October 2026:

- RevenueCat Expo setup: https://www.revenuecat.com/docs/getting-started/installation/expo

- RevenueCat React Native SDK: https://www.revenuecat.com/docs/getting-started/installation/reactnative
- RevenueCat web billing SDK and providers: https://www.revenuecat.com/docs/web/web-billing/web-sdk
- RevenueCat webhook security and retries: https://www.revenuecat.com/docs/integrations/webhooks
- RevenueCat webhook event types: https://www.revenuecat.com/docs/integrations/webhooks/event-types-and-fields
- RevenueCat pricing: https://www.revenuecat.com/pricing

E.4 v10.2 scope and verification caveat

The v10.2 SQL was a clean-install candidate, not an in-place migration, and was not executed in this environment. Before production, test on clean PostgreSQL; verify address joins, country constraints, triggers, provider-product mappings, webhook signatures and duplicate delivery, subscription-state reconciliation, active-member caps and concurrent quota enforcement. Create any migration from the actual deployed schema.

Appendix F. Historical v10.3 feed discovery, featured items and view tracking

This update extends the v10.2 baseline with an explicit feed-ranking strategy, separate featured-placement records, reusable per-user content view tracking and optional impression analytics. The companion file is `Diasporum_PostgreSQL_Schema_v10_3.sql`. This remains a clean-install candidate for PostgreSQL 15+ and is not an in-place migration.

F.1 View-tracking data contract

`user_content_views` stores one row for each `(user_id, content_type, content_id)`. The supported typed references in v10.6 are FEED_POST, EVENT, JOB, PROPERTY, VEHICLE, BUSINESS and SPARE_BAG_LISTING. The row records first_viewed_at, last_viewed_at and view_count. It is not an impression log: one view row is retained per user/content pair, and `view_count` advances only for qualified view events under a defined cooldown/idempotency policy.

The client reports visibility only after the configured viewport threshold (initial proposal: at least 50% visible for about one second) or when the full item/thread is opened. Backend API fetches, cache prefetch, and off-screen renders do not count. A batched view endpoint should validate the content reference, current permissions and target status, then perform a transactional UPSERT. Set `first_viewed_at` to the earlier timestamp, `last_viewed_at` to the later timestamp, and avoid decreasing timestamps when delayed requests arrive.

F.2 Featured placement data contract

`featured_items` stores the selected placement for a content item. `feature_kind` distinguishes EDITORIAL from PAID_PROMOTION; `placement_key` supports MAIN_FEED and later destination surfaces; `priority`, `status`, `starts_at` and `ends_at` control ranking and scheduling. `created_by_user_id` and `updated_by_user_id` provide actor audit. The referenced content remains canonical in its own domain table; duplicate title, price, venue or listing fields must not be copied into featured_items. Treat featured_items as the source of truth for scheduled feed placements; do not run a second promotion-ranking path from older `is_sponsored` flags. If legacy flags remain, keep them synchronized or stop using them for placement decisions.

The discriminator/reference pair intentionally uses a polymorphic reference rather than a direct foreign key. This is extensible but weaker than a true FK, so application services must validate target existence/type on create/update and before feed delivery. Do not trust stored feature status alone: each read must also verify target publication state, expiry/availability, country/community audience scope, blocking and moderation. Cleanup or lifecycle workers should end/cancel placements for removed content.

Where a paid promotion consumes a subscription quota or one-time purchase, record entitlement usage or billing-order-item targeting through the existing billing records. Payment confirmation, refund/cancellation and promotion

activation must be coordinated transactionally/idempotently. Editorial featuring can exist without a payment. Neither paid status nor a paid order expands the target content audience or skips moderation.

F.3 Optional featured impression analytics

`featured_item_impressions` records a qualified featured-card exposure, including featured item, nullable viewer, placement, one-based feed position, client/session identifier, event timestamp and a unique idempotency key. Record an impression when the card meets the qualified visibility threshold, not every feed response or off-screen render. Keep click-through, save, apply, RSVP and other actions in their appropriate event/engagement records; do not infer a click from an impression.

This table can grow quickly. Before production, select retention, aggregation and (if volume warrants) time partitioning based on measured traffic. Set a user-deletion/anonymisation policy and define access/retention for event-level analytics. The table is optional for the core feed algorithm; `user_content_views` is the source for seen/unseen state.

F.4 Feed assembly and promotion rules

- Filter candidates by authorization, current community/country, moderation, blocks, active publication status, content availability and dismissal rules before ranking.
- For You/default feed: unseen eligible content first, ordered newest first within the unseen group; previously viewed eligible content follows, also newest first. Meaningful updates may qualify content for resurfacing.
- Latest mode, if shipped, is reverse chronological among eligible candidates and is not re-ordered by view status.
- Interleave ordinary posts with relevant featured event/job/listing/business cards; do not make featured placement mandatory when no relevant eligible candidate exists.
- Initial experiment settings: first featured card after about 3-5 organic items, then another after 5-8 items if helpful; paid content about one in 8-10 feed items maximum near launch. Configuration/experiments can change these settings.
- Events use urgency and capacity; jobs use freshness and matching; property/vehicle use availability, location and preferences. Mark paid promotion clearly and cap repetition.

F.5 Static schema additions and caveats

v10.3 adds the `content_reference_type`, `featured_item_kind` and `featured_item_status` enum types; `user_content_views`, `featured_items` and `featured_item_impressions` tables; seven indexes for serving, point lookup and impression analysis; and an updated_at trigger for featured_items. Static checks do not prove that target content exists, that SQL executes on a live PostgreSQL server, or that concurrent view/feature updates are correct. Execute the script against a clean disposable PostgreSQL 15+ staging instance and run feed pagination, privacy, promotion entitlement, target-lifecycle and high-volume impression tests before production.

Appendix G. v10.5 schema update and change summary (historical baseline)

Historical status: the v10.5 changes below were incorporated into the v10.5 clean-install baseline. The v10.4 baseline remains unchanged. The current release candidate is v10.6, which is also a greenfield schema and not an in-place migration; do not apply it over a deployed database without designing and testing a migration from the actual deployed state.

G.1 Guides module removed

The v10.5 baseline removed the Guides feature; v10.6 retains that removal. Guide tables, review/revision/search/translation functions and triggers, guide community mappings, guide-specific type values,

guide media purpose and guide-target mappings are removed. Country practical categories map to FEED and EVENT only. The knowledge hub must not be advertised in current navigation, APIs or product scope.

G.2 Matchmaking actions: unmatch, cancel like and rewind

- `matchmaking_matches.status` supports MATCHED, UNMATCHED, CLOSED and BLOCKED. Unmatch records the closing user and timestamp/reason; the service must close or disable the linked direct conversation in the same transaction and deny future messages.
- `matchmaking_swipes` stores the current decision and an ACTIVE, WITHDRAWN or REWOUND state. Mutual matching is allowed only when both directional rows are ACTIVE LIKE.
- `matchmaking_swipe_history` is an append-only event log for LIKE/PASS swipes, LIKE_CANCELLED and REWOUND reversals. A unique reversal index prevents reversing one swipe event twice.
- Cancel like is allowed only for the user's own active outgoing LIKE before a current mutual match exists. Rewind may reverse only the actor's latest eligible, unreversed swipe; the current-state update and reversal event must be atomic and idempotent.
- Blocks and moderation restrictions take precedence over likes, cancellation, rewind, matching and messaging. The schema keeps one match row per unordered pair; re-matching an unmatched pair is not supported without a later policy/schema change.

G.3 Property and vehicle JSONB attributes

- Both `property_listings` and `vehicle_listings` include `additional_attributes jsonb NOT NULL DEFAULT '{}'::jsonb`, with a database check that the value is a JSON object.
- Spring Boot must validate keys, value types, ranges, allowed values and maximum payload size against the listing category (property type or vehicle category) before insert/update. Reject unknown or unsupported fields; do not trust the client payload.
- The API displays only attributes allowed for that category. Filtering by JSONB attributes is permitted only where the category schema explicitly marks the attribute as filterable. GIN indexes support suitable containment predicates.
- Frequently filtered fields remain dedicated columns: price, property bedrooms, vehicle mileage, `model_year` and `registration_year`. JSONB is for flexible category-specific details, not a replacement for indexed core fields.

G.4 Vehicle listing fields

- `drive_type` is a controlled text key: FWD, RWD, AWD, 4WD, OTHER or UNKNOWN.
- `previous_owners` is a nullable non-negative small integer.
- `model_year` is a nullable small integer with the schema's configured valid year range. `registration_year` remains a separate nullable field, because the model year and first registration year are not equivalent.
- `insurance_group` is deliberately not present in the v10.5 schema.

G.5 Matchmaking profile preference cleanup

The following superseded fields are not part of the v10.5 matchmaking schema: `residency_type`, `ethnicities`, `family_involvement`, `preferred_residency_types`, `specific_country_codes`, `expand_search_when_limited`, `ethnicity_preferences`, `preferences_version` and `location_mode`. Do not recreate them in the profile or preference tables.

G.6 Feature access policies remain distinct from rollout flags

- `feature_flags` is for global/country/community feature rollout only.
- `feature_access_rules` separately defines FREE, PREMIUM and ROLE access. It supports global, country or community-specific scope, optional usage limits/reset periods and role-specific policies.
- The backend resolves community rules before country rules before global rules, validates subscription/plan entitlements and roles, and enforces usage limits for every protected operation. Database rows are configuration; they do not replace endpoint authorization.

G.7 Clean-install verification and release gates

- Run the v10.8 strict clean-install SQL only against a brand-new empty PostgreSQL 15+ staging database with PostGIS installed. v10.4–v10.6 remain unchanged historical baselines.
- Verify that no Guides tables, Guide-specific enum labels or Guide-specific functions/triggers remain; verify the nine removed matchmaking fields and `insurance_group` are absent.
- Test mutual LIKE creation, cancel-like before match, rewind of the latest eligible swipe, rewind rejection for older/already-reversed swipes, unmatch, block precedence, and denial of messages after unmatch.
- Test JSONB category validation, unknown-key rejection, typed display, filterable-key allow-lists, and query plans for core columns and supported JSONB containment filters.
- The v10.10 package receives static source validation and Word rendering review only; the SQL has not been executed against a PostgreSQL server in this environment. Live staging execution remains mandatory.

Area	v10.6 change
Guides	Removed from current product scope and SQL schema, including guide-specific dependent objects.
Matchmaking	Added unmatch status, outgoing like withdrawal, rewind state and append-only swipe history.
Listing attributes	Added object-shaped JSONB attributes to property and vehicle listings, plus GIN indexes and backend validation contract.
Vehicle listings	Added drive_type, previous_owners and model_year; retained registration_year separately; omitted insurance_group.
Matchmaking preferences	Confirmed nine superseded preference fields are absent.
Feature access	Added scoped FREE/PREMIUM/ROLE feature_access_rules independently from feature_flags.
Direct messaging	Replaced DIRECT_MESSAGE_START with DIRECT_MESSAGE_SEND. Quota scope is PER_RECIPIENT; each successfully accepted outgoing message consumes one unit for that recipient, independently per sender.
Quota semantics	quantity_limit NULL means unlimited when the entitlement is enabled; 0 means no usage is allowed; positive values are caps. reset_period is configurable per plan entitlement.
SAVE_CONTENT	Deferred for a later schema update; intentionally not present in v10.6.

Appendix H. v10.6 billing quota behaviour and deferred feature

H.1 Direct-message send quota

The current billing feature is DIRECT_MESSAGE_SEND. It counts outgoing direct messages successfully accepted by the service, not conversation starts. The quota scope is PER_RECIPIENT: the same sender has a separate allowance for each recipient, and messages to that recipient across multiple conversations share the allowance. Incoming replies do not consume the sender's quota.

For each accepted outgoing message, write an idempotent billing_usage_records usage unit in the same service transaction as message acceptance, with user_id set to the sender, target_type set to USER and target_id set to the recipient profile ID. Do not count failed sends, duplicate retries, inbound messages or opening a conversation. Use the target-specific usage index for checks. A sender's allowance to Alice must not affect the allowance to Bob.

H.2 Plan-configurable limit and reset period

Configure quantity_limit and reset_period on billing_plan_entitlements for each plan. Supported reset periods are DAY, WEEK, MONTH, YEAR, SUBSCRIPTION_PERIOD and NEVER. The reset window must be resolved from the active plan entitlement; application code must not hard-code the window. The schema requires a reset_period whenever quantity_limit is non-NULL. billing_features.reset_period is NULL for DIRECT_MESSAGE_SEND because the effective period is plan-specific.

When the entitlement is enabled, quantity_limit = NULL means unlimited usage; quantity_limit = 0 means no usage is permitted; a positive number is the maximum outgoing-message count per sender-recipient pair for the configured reset window. An entitlement with enabled = false denies the feature regardless of quantity_limit. If no applicable plan entitlement is configured, the backend must fail closed rather than infer a default. The schema does not seed an arbitrary quota or reset period; administrators must configure and test the intended free and paid plan values before enabling billing enforcement.

When a user reaches the configured limit, do not silently charge them. Return a clear entitlement/paywall response and allow messaging only after the user has an eligible subscription or completes an explicitly confirmed, supported purchase. Verify billing events using the trusted server-side billing flow and make the usage/purchase operation idempotent.

H.3 Deferred feature — SAVE_CONTENT

SAVE_CONTENT is recorded as a future requirement for normal/personal users to save or bookmark supported content such as feed posts, property listings, vehicle listings and other approved content types. It is deliberately not included in the v10.6 billing feature catalogue or schema. Add it only in a later approved schema update.

H.4 v10.6 validation gates

The v10.10 smoke test checks Follow/mutual connection integrity, persistent voice rooms and session comment purge, moderation restrictions, poll rules, featured placement allow-lists, Business Account permissions/lifecycle, push-delivery records, service marketplace constraints and PostGIS discovery.

Appendix I. Historical v10.7 service marketplace, messaging contexts and temporary voice comments (superseded where inconsistent with Appendix K)

I.1 Scope and non-goals

The service marketplace is a lead-generation/contact feature. Customers publish requests for services; registered providers discover requests, send responses or quotes and access a customer through in-app chat or separately revealed contact details. Actual arrangements, booking, work completion and payment occur off-platform. No `service_provider_locations` or `service_provider_coverage` tables, booking tables, appointment scheduler, service checkout, escrow or service-payment ledger are included.

I.2 Service marketplace logical data model

`service_categories` and `service_category_translations` define a configurable, translated catalogue using the existing language catalogue. `service_providers` is owned by exactly one `profiles` row or one `business_accounts` row and has a single `base_location geography(Point,4326)` and editable `radius_km`. `service_provider_services` maps providers to categories. `service_requests` records customer, category, country, approximate service-area point, optional address, budget/currency, preferred date, status and expiry. `service_request_media` stores approved media references. `service_request_responses` stores one provider response per request (interest/contact/quote) with an auditable acting user. `service_lead_access` records the one provider/request access grant. `service_reviews`, `service_reports` and `service_moderation_actions` provide trust and moderation support.

Use GiST indexes for provider `base_location` and request `location`; discover candidates with `ST_DWithin(request.location, provider.base_location, radius_km * 1000)` and sort by `ST_Distance`. Restrict results to active/open eligible records, matching country/category and privacy scope. Never expose exact addresses or contact information in the broad discovery result. Providers must set a valid base location; absence must not be interpreted as worldwide coverage.

I.3 Billing and idempotent lead access

Use the shared `billing_features`, `billing_plan_entitlements` and `billing_usage_records`; do not create a second billing engine. `SERVICE_JOB_POST` controls request-publishing price/quota; `SERVICE_LEAD_ACCESS` controls contact-detail unlock and/or initiating in-app chat. `SERVICE_QUOTE_SEND` is a future-compatible optional feature that remains disabled until deliberately configured. Do not add `SERVICE_PROVIDER_REGISTER`. The schema must not hard-code product price, plan quota or reset period. Disabled/missing configuration fails closed.

`service_lead_access` has a unique `(service_request_id, service_provider_id)` key and can hold both `chat_enabled` and `contact_details_unlocked`. Grant/update this record and create exactly one corresponding billing usage record in the same transaction using a stable idempotency key. `billing_usage_record_id` is required and immutable for an existing lead grant. Upgrading a grant from chat to contact details—or the reverse—must reuse the existing access and billing usage record and must not create a second charge. Unique constraints protect duplicates; the service must handle concurrent attempts and retry safely.

I.4 Reviews, reporting and moderation

No booking is required, but a review requires a recorded interaction between the request customer and the provider (lead access or a response). It is not labelled as proof of completed work. Self-review by the request customer acting as provider owner/business member is rejected. Service reports target a request, provider or review; the polymorphic target type/ID must be validated. Report status/assignment/resolution is separate from `service_moderation_actions`, which records admin actions and reason/metadata. Apply rate limits and preserve audit records under the documented retention policy.

I.5 Messaging contexts and request lifecycle

The eight supported conversation contexts are `GENERAL`, `PROPERTY_LISTING`, `VEHICLE_LISTING`, `BUSINESS_PROFILE`, `EVENT`, `JOB_LISTING`, `SPARE_BAG_CONTACT` and `SERVICE_REQUEST`. `FEED_POST` is intentionally excluded; `MATCHMAKING_MATCH` is intentionally excluded and uses `GENERAL`. Context-specific uniqueness uses ordered participant identities plus context type and context ID. Same pair + same listing reuses a thread; same pair + different listing/job creates a separate thread. A separate GENERAL conversation can exist alongside all contextual threads. Context references are polymorphic UUIDs and must be validated against their respective resource table and current publication/eligibility state.

Context-based enquiries can be accepted immediately only after the resource permits contact and privacy, block, provider lead access and other entitlement checks pass. `GENERAL` conversations follow recipient `dm_permission`; friend or mutual-follower paths may proceed when allowed. Otherwise, when policy permits requests, create a REQUESTED conversation and one message request. The request may contain one initial message; no further message can be sent until accepted. Decline is request-scoped, not a permanent user-pair block; blocking is pair-wide and overrides all contexts. Add `MUTUAL_FOLLOWERS` to the accepted privacy policy values and evaluate both directions.

Conversation participants may be personal users or Business Accounts. A business conversation side is associated with its active accepted members; the actual sender is the acting user, and an optional business-account ID records that the member acted for the business. Recheck membership/role and both sides' privacy on every operation; when membership is revoked, remove that member's access while preserving the shared thread. Never serialize private business/customer contact fields without the appropriate grant.

When a matchmaking pair becomes MATCHED, the same transaction creates a friendship linked to that match and creates/reuses an accepted GENERAL conversation. The chat UI derives a Matched badge from the active match record. On unmatch, delete only the friendship created by that match; preserve a friendship that existed independently before matching, keep the ordinary GENERAL conversation/history and remove the badge.

I.6 Voice-room comments are ephemeral

`voice_room_messages` stores comments by `session_id`, not merely room ID. Only an active session participant may add a comment, and only while the session status is ACTIVE. Insert the durable comment and a `VOICE_ROOM_COMMENT_CREATED` outbox event in one transaction; the worker publishes to the authorised Ably session channel. Clients must fetch current history through the API rather than treating realtime delivery as the durable source of truth.

When a host disconnects and a session is temporarily INACTIVE/within grace, keep comments for possible recovery. On a definitive session transition to ENDED or EXPIRED, delete all session comments and pending undelivered comment-created outbox events, then enqueue `VOICE_ROOM_COMMENTS_CLEARED` so clients clear cached/in-memory comments. Closing/expiring the room closes its associated live/inactive session so the cleanup trigger runs. A new session starts with no comments. Moderation/audit actions remain separate. Test comment/session-close concurrency and the publisher/clear-event ordering in the application.

I.7 v10.8 verification limits

The v10.8 deliverables were statically inspected and smoke-test assertions were written; they were not executed against a live PostgreSQL/PostGIS server in this environment. Before release, run the strict clean-install SQL on a fresh PostgreSQL 15+ database with PostGIS available; execute smoke tests and automated API/service integration tests for the paths in Sections 23.2–23.7. Review DDL/function behavior, concurrency, query plans, event retries and data-retention cleanup before production. The schema is a clean-install candidate, not an in-place migration.

Appendix J. Historical v10.8 shared PostGIS distance and nearby discovery (superseded where inconsistent with Appendix K)

Version 10.8 extends the v10.7 baseline with spatial support for nearby people, matchmaking, jobs, property and vehicle listings, events, business branches, airports and the existing service marketplace. PostGIS is used for geography-aware distance and radius queries; a location column or index does not replace application-level permission checks.

J.1 Shared spatial model

- `addresses.location geography(Point,4326)` is the exact point synchronized from numeric coordinates. `addresses.discovery_location geography(Point,4326)` is a separate approximate/public point with a GiST index; public discovery queries use it, never the exact point.
- `airports.location geography(Point,4326)` is synchronized in the same way and indexed for future airport-proximity queries.
- `user_discovery_locations` is a distinct opt-in table for an approximate point used by nearby-people discovery and/or matchmaking distance checks. Keep the point only while at least one such use is enabled and consented; delete it when the user opts out of all location-based discovery. It is never a substitute for a private postal/home address.
- `service_providers.base_location` and `service_requests.location` remain independent geography points with existing GiST indexes and provider radius matching.

J.2 Distance calculation

Use ST_DWithin(geography, geography, radius_metres) for radius filters and ST_Distance(geography, geography) for distance sorting. Convert kilometres to metres before the ST_DWithin call (radius_km * 1000). Keep the origin point as a bound query parameter rather than injecting it into SQL text.

```
SELECT j.id, j.title, ST_Distance(a.discovery_location, :origin) / 1000.0 AS distance_km
FROM job_listings j
JOIN addresses a ON a.id = j.workplace_address_id
WHERE j.status = 'PUBLISHED'
  AND a.discovery_location IS NOT NULL
  AND ST_DWithin(a.discovery_location, :origin, :radius_km * 1000)
ORDER BY ST_Distance(a.discovery_location, :origin) ASC
LIMIT :page_size;
```

J.3 Feature behaviour

Feature	Rule
Nearby people	Use opted-in approximate discovery points; enforce profile visibility, allow_people_suggestions, account status, community/audience scope and reciprocal block exclusions. Never return raw location points.
Matchmaking	Compare two users' discovery points with ST_Distance and enforce max_distance_km bilaterally. If either user requires a distance limit but a point is missing, that pair fails distance compatibility.
Jobs	Filter published jobs by workplace address location only for physical/field-based workplaces; remote jobs are not nearby physical jobs.

Feature	Rule
Property and vehicle listings	Radius-search against referenced addresses' `discovery_location` only where a suitable point is available. If exact address details are private, store/use an intentionally approximate locality point for public discovery.
Events and business directory	Use the spatial point of the event venue or public business branch. Respect the entity's publication state and location visibility.
Service marketplace	Continue to match open service requests to active providers using ST_DWithin(request.location, provider.base_location, radius_km * 1000), category/country filters and ST_Distance ordering.

J.4 Privacy and REST API contract

- Obtain explicit consent before storing an approximate discovery location. Retain it only while at least one consented location-based feature needs it; delete the stored point when the user opts out of all location-based discovery. A disabled surface must not consume the point.
- Never serialize PostGIS point objects, exact latitude/longitude or private address components in discovery responses. Return a rounded distance such as "within 5 km" or a suitably rounded `distanceKm` only when appropriate.
- For public listing discovery, use `addresses.discovery_location`, not `addresses.location`. Do not calculate or expose distance from a confidential exact home address; use a rounded/blurred locality point.
- All nearby queries must also enforce lifecycle, country/community/audience, visibility, deletion, blocking and applicable feature-entitlement rules. Location proximity by itself is not permission to view a person or listing.

J.5 Spring Boot module placement

Keep reusable technical primitives under platform.geospatial (for example GeoPoint, DistanceUnit and small PostGIS utilities). Feature-specific spatial queries remain in each module's infrastructure.persistence.spatial package: identity/social discovery, matchmaking, jobs, property listings, vehicle listings, events, business directory and service marketplace. Do not centralise all business search rules into the platform module.

J.6 Validation and release gates

- Run strict clean-install SQL only against a disposable empty PostgreSQL 15+ database with PostGIS available, then execute the v10.10 smoke test.
- Test address coordinate insert/update/clear behaviour, direct location-update attempts, opt-in/opt-out deletion, bilateral matchmaking radius constraints and all visibility/block cases.
- Run EXPLAIN (ANALYZE, BUFFERS) using realistic location distributions to confirm GiST-assisted query plans for each feature. An index definition alone does not prove it is used.
- The precision_radius_m field records intended approximation precision; the backend must actually produce an approximate point before storing it.
- This package was statically reviewed; it has not been executed against a live PostgreSQL/PostGIS server in this environment.

Appendix K. v10.9 consolidated batch update

This appendix is the authoritative v10.9 interpretation where older historical appendices describe behavior superseded by the consolidated batch. v10.9 is a clean-install baseline, not an in-place migration. Historical appendices remain as a record of prior iterations; they must not override this appendix or the v10.10 SQL.

K.1 Social graph and matchmaking separation

`user_follows` is the only persisted directional follow edge. The `user_mutual_connections` view derives a mutual connection where reciprocal follow rows exist. v10.9 removes `friend_requests` and `friendships`, and changes direct-message policy vocabulary to `MUTUAL_FOLLOWERS`. The backend enforces active profiles, the recipient's `allow_followers` setting, blocks in either direction, and rate limits. The current privacy setting is an allow/deny switch, not a follow-request/acceptance workflow; do not represent it as private-account approval.

A mutual matchmaking LIKE creates exactly one `matchmaking_matches` row and a normal accepted GENERAL conversation. It does not create, remove or imply follows. Unmatching closes/blocks the match according to status policy while preserving message history and independent follow edges. Messaging authorization remains independent and is enforced on every API operation.

K.2 Persistent social-audio rooms

- `voice_rooms` is durable metadata. Keep rooms discoverable while active/locked according to visibility and `is_discoverable`, even when no live session exists.
- `voice_room_sessions` represents each distinct live event. Only one ACTIVE/INACTIVE (recoverable) session is permitted per room. Terminal sessions may be archived as history, but they do not expire or delete `voice_rooms`.
- The host/inactivity recovery grace period is 30 minutes by default. A background worker must reconcile heartbeat and provider state, move a disconnected session to INACTIVE, restore it during the grace period, or terminally end/expire it when the grace expires.
- Durable room owner and session HOST are separate roles. Stage roles include HOST, MODERATOR, SPEAKER and LISTENER. Backend operations control invites, speaker requests, seats, mute/remove, host transfer, locking, room archive/suspension and moderation audit.
- Capacity checks must be serialized on the session row or equivalent lock to avoid concurrent over-admission or excess speakers. Reconnect updates the existing active participant and must not create duplicate active participation.
- Comments are tied to one session. They remain recoverable during INACTIVE grace; when a session reaches ENDED/EXPIRED, delete comments and undelivered comment-created outbox rows, then emit `VOICE_ROOM_COMMENTS_CLEARED`. The client clears cached comments and ignores stale events for terminal sessions.
- PostgreSQL stores room/session state; Ably distributes presence and authorized real-time events; an RTC/SFU provider carries audio. Keep the provider decision open between LiveKit and Cloudflare Realtime SFU until evaluated in production-like tests. Do not put audio transport in the Expo client itself.

K.3 General chat restrictions

`user_mutes` supports scoped restrictions with action `MUTE` or `BAN`, optional room or community scope, reason, start/end times, revocation and an audit entry. Room scope is allowed only for CHAT and is constrained to a specific room. A mute prevents sending while reading may remain allowed; a ban can additionally block join/history access according to the application policy. The database guard blocks new CHAT/ALL-restricted message inserts. Spring Boot must also enforce all read/join/send policies and moderator authority.
`chat_room_user_state.notifications_muted_until` is self-service notification preference only.

Moderator permission checks must use active community membership with MODERATOR/COMMUNITY_ADMIN role for scoped actions; global restrictions require an explicitly authorized platform-moderation context. Every

enforcement path must check expiry and revocation. The room policy/audit model must not rely on client button visibility.

K.4 Poll authoring, voting and privacy

- A poll is represented by `feed_posts` with `post_type=POLL`, `feed_polls`, `feed_poll_options` and `feed_poll_votes`. Configure `max_options` in the range 2–20 (default 10); published polls require at least two options.
- Creation contract: insert the post as DRAFT; create the `feed_polls` row and at least two options; then publish the post. A direct insert/update of a published POLL without its complete poll setup is rejected.
- Voting validates poll openness and option count; advisory locking plus the poll row lock protects competing votes. Single-choice duplicate ballots are rejected. `allow_vote_change` controls changing an existing selection; `allow_vote_withdraw` controls user-initiated deletion before close.
- Once voting starts, option text/order cannot change and no options may be added or deleted. Poll voting/privacy policy and max_options are also frozen after the first vote; max_options cannot be lowered below the current option count. Retain the option set so percentages and previous votes remain meaningful. Parent poll/post deletion for an authorized retention operation is a cascade, not user withdrawal.
- `closed_at` permits authorized early closing; an elapsed or explicitly closed poll cannot be reopened by extending `closes_at`. A poll closes for voting once `closed_at` is set or `closes_at<=now()`.
- `results_visibility` is `IMMEDIATE`, `AFTER_VOTING` or `AFTER_CLOSE`. The API filters results for each viewer accordingly. `anonymous_voting=true` means poll viewers/creators do not receive voter identities; votes still reference user IDs internally for integrity, abuse prevention and vote policy. This is not cryptographic anonymity from the platform operator.
- Calculate counts and percentages from vote rows unless measured performance justifies a carefully maintained counter. Return total voters and vote counts consistently; define “voters” separately from total choices for multi-select polls.

K.5 Featured-item placements

`featured_placement_rules` is a backend-controlled allow-list of content-type/placement pairs, such as MAIN_FEED and domain-specific EVENTS, JOBS, PROPERTY_LISTINGS, VEHICLE_LISTINGS, BUSINESS_DIRECTORY and SPARE_BAG. The user may buy/request an eligible campaign, but cannot select any arbitrary placement or bypass the server policy. Validate content existence/type, publication state, current availability/expiry, country/community audience, moderation, entitlement, promotion product, payment and campaign window at creation and delivery time.

Create one `featured_items` row per placement. If a campaign targets multiple surfaces, create distinct placement rows with a shared campaign key and consistent billing/analytics; do not assume one row silently represents multiple surfaces. The polymorphic `content_id` still requires application-level type/existence checking because it cannot have a conventional FK to multiple domain tables.

K.6 Business Accounts: lifecycle and authorization

- A Business Account is a workspace, not a login. Its creator becomes its first accepted OWNER. Active or suspended accounts must have one accepted active OWNER; a partial unique index prevents two active OWNERs and deferred constraint triggers enforce that an owner remains at transaction commit.
- Membership identity (account/user) is stable. Invite, accept, expire, revoke and role changes are audited. Re-invites/rejoins reactivate an existing relationship under policy rather than creating ambiguous duplicate membership history.
- Use `business_role_permissions` as the permission matrix and `business_account_has_permission(account,user,permission)` (or a centralized equivalent) to verify active account, accepted/unrevoked membership and role permission on protected requests. The table is a policy input; API authorization is still mandatory.

- `OWNER`, `ADMINISTRATOR`, `EDITOR`, `LISTING_MANAGER` and `BILLING_MANAGER` have different least-privilege grants. In particular, listing managers do not inherit billing or ownership transfer, and billing managers do not automatically gain listing management.
- Business profiles/branches/locations/opening hours remain separate from the account workspace. Eligible content has exactly one owner context (personal user or Business Account); `created_by_user_id` identifies the human actor.
- Closing an account records `closed_at`/reason, revokes all active member access and writes audit events. Reopening must be explicit, authorized and transactional, reactivating the intended owner/member state before account status becomes ACTIVE/SUSPENDED.
- Membership access to business conversations is synchronized for UX but always revalidated by Spring Boot on read/send/subscribe/attachment operations. Prevent cross-account IDOR, search leakage and data access after role revocation.
- Business subscriptions and one-off listing/promotion charges are separate. One business subscription is account-level and may cover eligible profiles/branches subject to plan caps. Verification status is never inferred from payment/subscription state.

K.7 Expo push delivery and in-app notifications

Spring Boot creates durable `notifications` records. An outbox event fans out push work after commit. `device_tokens` tracks provider/platform, active state, registered/last-used/invalidation times and device/app metadata. `notification_deliveries` stores a unique notification/token attempt with retry timing, provider receipt and error detail. The worker should check `notification_preferences`, deduplicate, retry retryable failures with backoff, reconcile receipts and deactivate invalid tokens. Expo Push is the initial mobile option to evaluate; APNS/FCM may remain provider values only if directly needed. Ably is for connected real-time updates and is not a substitute for push alerts when the app is backgrounded/closed.

K.8 Listing, country and location decisions

- Spare Bag `listing_country_id` is removed. Derive the listing's market/origin country from `origin_airport_id` → `airports.country_id`; keep `community_id` as target-audience selection and validate community destination constraints. Other listing domains retain their own market/location country fields where those describe the listing rather than only audience.
- Vehicle listings retain dedicated `model_year`, `registration_year`, `drive_type` (including `FOUR_WD`), `previous_owners`, and JSONB `additional_attributes`. Do not add `insurance_group`.
- Use the shared PostGIS address model and geography points/GiST indexes. Public discovery uses a separate approximate/opt-in point; exact private addresses are never exposed by ordinary listing/profile DTOs.
- Service providers retain one base location and `radius_km`; do not add separate service-provider-location or coverage-area tables under current scope. The service marketplace remains lead/contact based—no booking, escrow or in-app service payment.

K.9 Billing, feature flags and service boundaries

- Personal plans are FREE/SILVER/GOLD; business plans are BUSINESS_FREE/BUSINESS_SILVER/BUSINESS_GOLD. `quantity_limit=NULL` means unlimited only when enabled; zero means no allowance; positive values mean a quota.
- Distinguish consumable/resetting quotas from concurrent caps such as team seats and business locations. `DIRECT_MESSAGE_SEND` is scoped per sender-recipient pair and uses configurable reset periods. `SAVE_CONTENT` remains deferred.
- `feature_flags` controls rollout/availability. `feature_access_rules` and subscription entitlements control eligibility and quotas. Do not use one mechanism as a substitute for another or for authorization.
- The backend remains a Spring Boot REST API modular monolith. Next.js is the web client; Expo/React Native is mobile; PostgreSQL/PostGIS is the source of truth; Cloudflare R2 stores bytes; Ably handles realtime; Keycloak or Authentik is the OIDC provider to finalize.

K.10 v10.9 release acceptance checklist

1. Build the strict clean-install SQL on a fresh PostgreSQL 15+ database with PostGIS in a disposable staging environment and verify the transaction commits cleanly.
2. Verify there are no `friend_requests`/`friendships` tables or matchmaking friendship triggers; verify `user_mutual_connections` derives reciprocal follows and block cleanup removes both directional edges.
3. Verify a voice room remains when its session reaches ENDED/EXPIRED, a new session has a new ID, comments survive the recovery grace and are purged with a `VOICE_ROOM_COMMENTS_CLEARED` event at terminal close.
4. Race-test session joins, host transfers, speaker promotions, seat assignment, follow-vs-block operations and single-choice votes using concurrent transactions.
5. Test chat mute/ban expiry and revocation, moderator scoping, global-mod authorization, room-specific limits and backend checks on send/join/read/history.
6. Test poll draft assembly/publication, min/max options, vote change/withdraw rules, anonymous result DTOs, early close and rejection of deadline reopening.
7. Test featured placement allow-list rejection and verify the API ignores a client-supplied unsupported placement; verify promotion eligibility is checked on serving as well as creation.
8. Test Business Account initial owner, concurrent owner transfer, invite expiry, permission matrix, membership revocation, closure/recovery, associated conversation access and billing/verification independence.
9. Test Expo token rotation/invalidation, notification preference filtering, outbox idempotency, retry backoff, receipt reconciliation and invalid-token cleanup.
10. Test Spare Bag origin-country derivation, community audience constraints and PostGIS distance sorting/privacy. Validate every documented DB invariant against the actual staging catalog.

K.11 Validation status and migration warning

The v10.10 SQL is a greenfield clean-install baseline. It must not be run over an existing v10.8 or v10.9 database. Existing deployments need a separately reviewed data migration with a schema inventory, backfill, duplicate/invalid data checks, rehearsed rollback or forward-fix path and verified backups. In this package, SQL was statically inspected, version/object/reference assertions were run, and the DOCX was rendered for layout QA. A live PostgreSQL/PostGIS execution was not available; staging execution remains a release gate.

Appendix L. v10.10 Reels short-video feature

This appendix is the authoritative Reels addition to v10.10. Reels use the existing feed and media model. It does not introduce a separate social graph, a per-Reel Nearby setting, a filming-location field, a reel_details table, or a dedicated Nearby opt-in setting. This remains a clean-install schema baseline, not an in-place migration; deployments built from an earlier v10.8 or v10.9 package need a separately reviewed migration for these schema deltas.

L.1 Product and feed decisions

- A Reel is a feed_posts row with post_type = REEL. The existing post is the source of truth for creator, community, country/audience, caption, status, moderation, comments and publication lifecycle.
- The video is attached through feed_post_media and described by media_objects. A Reel must have exactly one creator-owned media_objects record with purpose = FEED_VIDEO. No separate reel_details or Reel-specific engagement tables are needed for the initial release.
- Provide three feed modes: Community, Nearby and Following. All modes reuse the same post, comments, reactions, shares, blocks, moderation, views and analytics.
- No AI recommendation service is required for launch. Community and Following are chronological; Nearby is ordered by address-to-address distance with publication time as a deterministic tie-breaker.

L.2 Current address and Nearby semantics

- Add profiles.current_address_id as a nullable FK to addresses.id. It identifies the profile's current address, not a filming location. It must reference an address owned by that same profile, with coordinates and a country matching profiles.current_country_id. Profiles without a usable current address can use Community and Following but are omitted from Nearby.
- Nearby distance is computed between the viewer's current address point and the Reel creator's current address point using PostGIS ST_Distance. Do not use a Reel location, filming location, current city text, or user_discovery_locations as a substitute.
- Use addresses.location for this narrowly-scoped authenticated server-side ranking query and add a GiST index on addresses.location. This is a privacy-sensitive internal use: feed DTOs must not include either address, formatted address, raw coordinates or numeric distance; access logs and analytics must not record exact points.
- Existing addresses.visibility and entity-level access rules still protect address retrieval. The Nearby service uses the address only to calculate ordering; it does not grant the viewer access to the address record. Do not add an allow_my_reels_in_nearby setting, use_current_address_for_nearby_reels setting, or include_in_nearby column. Community visibility, blocks, moderation and active-account checks determine content eligibility.

L.3 Data model changes

The following changes are required in the v10.10 clean-install schema. These reuse the current feed and media relationships; no separate Reel-detail or engagement table is introduced.

- Add REEL to feed_posts.post_type and allow an empty caption for this post type while preserving required text for other post types.
- Add published_at to feed_posts. Set it at first publication, preserve it across ordinary edits/restoration, and use it with id as a stable chronological cursor key.
- Add nullable profiles.current_address_id referencing addresses.id. Triggers must enforce that the address belongs to the profile, matches current_country_id, has usable coordinates, and cannot be de-owned or made inconsistent while selected.
- Reuse feed_post_media and media_objects. Publication requires exactly one creator-owned FEED_VIDEO in READY state; database guards reject non-video/multiple/foreign media and prevent a published Reel from losing its valid video.
- Add a GiST index on addresses.location and partial feed indexes for published Reels by community and author. Exact points are for server-side Nearby ordering only.

No reel_details table, per-Reel include_in_nearby field, separate Nearby opt-in setting, or filming-location field is included.

Object	Change	Purpose
feed_posts	Add REEL post_type and nullable published_at; allow an empty body only for REEL captions.	Shared post, audience, creator, moderation and feed-order source of truth.
profiles	Add current_address_id FK and current-address integrity validation.	Resolve viewer and creator current addresses.
addresses	Add GiST index on exact location.	Efficient internal distance/radius predicates for Nearby.
feed_post_media	Reuse; triggers enforce one video for REEL.	Attach the one canonical video asset.
media_objects	Reuse purpose/status/owner/duration/dimensions and media processing pipeline.	Video ownership, upload, processing readiness and playback metadata.
Existing social tables	No new duplicates.	Reuse comments, reactions, shares, follows, blocks, communities, views, analytics and reports.

L.4 Reel authoring and publication lifecycle

- Create a REEL feed_posts row as DRAFT. A blank caption is supported; other feed post types retain their non-empty body rule.
- Attach exactly one media asset through feed_post_media. The asset must be owned by the author and have purpose = FEED_VIDEO. Upload/processing may remain pending while the post is DRAFT.
- Publish only when the attachment exists, the media asset is READY and not deleted, and no second or non-video attachment exists. Database triggers enforce the publication boundary and prevent replacing/detaching the video of a published Reel. Hide the Reel before changing its video or taking the asset out of READY state.
- feed_posts.published_at is set on first publication and is preserved through ordinary edits, hiding and restoration. It is used for chronological ordering and cursor pagination. Use a stable (published_at DESC, id DESC) key.
- Video processing continues through the existing R2/media_objects/media_processing_jobs pathway. Before production, define maximum upload size/duration, allowed codecs/container, thumbnail/transcode profile, playback authorization, retention and egress-cost monitoring.

L.5 Reels feed modes

Mode	Eligibility	Ordering
COMMUNITY	Published Reels visible to the selected active community or permitted country-wide audience; exclude deleted, hidden, blocked or otherwise ineligible content.	published_at DESC, id DESC
NEARBY	Published eligible Reels whose creators have a valid current address; apply the same community/audience, block, account and moderation checks. Optional radius is a query filter, not a stored per-Reel setting.	ST_Distance(viewer current address, creator current address) ASC, published_at DESC, id DESC
FOLLOWING	Published eligible Reels authored by users followed by the viewer, subject to the normal audience and safety checks.	published_at DESC, id DESC

API contract candidate: GET /api/v1/reels/feed?mode=COMMUNITY|NEARBY|FOLLOWING&cursor=...&limit=... with optional radiusKm for Nearby. Return the Reel/post ID, author display information, caption, authorized playback information, comment/reaction state and opaque next cursor. Never return the current-address FK, address IDs, exact points, formatted addresses or numeric proximity distance.

L.6 API and application module responsibilities

- Keep implementation in the existing feed module, with a reels subpackage for Reel commands, feed mode queries and ranking. The media module owns uploads, asset metadata, processing jobs, playback grants and media lifecycle.
- Suggested endpoints: POST /api/v1/reels (create draft), POST /api/v1/reels/{id}/publish, GET /api/v1/reels/feed?mode=..., and a view-event endpoint. Reuse existing post comment/reaction/report endpoints when practical.
- The Nearby repository joins viewer profiles.current_address_id -> addresses.location and each candidate author profiles.current_address_id -> addresses.location. Use ST_Distance for sorting and ST_DWithin when a radius is supplied. Use cursor pagination and test query plans with realistic data.
- Record impressions, watch duration and completion through existing analytics_events and, where useful, user_content_views. Add no AI ranking infrastructure in this release. Avoid counting every autoplay start as a qualified view; make events idempotent and filter client-generated noise.

L.7 Reels acceptance checks

- A REEL post can exist as DRAFT before media is READY; publication without exactly one valid READY FEED_VIDEO is rejected.
- Reject image attachments, multiple attachments, a video owned by another user, and replacing/removing a published Reel's video. Hidden content can be moderated before changing its media lifecycle.
- A current_address_id can only reference an address owned by the profile, with matching current country and usable coordinates. Address ownership/country/coordinates cannot be changed into an inconsistent state while still selected as current.
- Nearby uses viewer-to-creator current-address distance, never video/filming location. Feed responses/logs contain no raw addresses/coordinates/numeric distance. Users without a usable current address do not receive Nearby results.
- Test Community, Nearby and Following feed audience eligibility, mutual blocks, moderation, pagination stability, absent address and no-candidate fallback, optional radius, and ordering ties.
- The current clean-install SQL and strict clean-install SQL contain equivalent schema bodies. Execute the strict script and smoke tests on a disposable PostgreSQL/PostGIS instance before claiming live validation.

Appendix M. v10.11 Community Help & Offers

This appendix is the authoritative Community Help & Offers addition to v10.11. It is a structured feed feature for informal assistance, donations, language practice, newcomer support and volunteering—not a separate marketplace. Paid professional services remain in the Service Marketplace. The database adds one lifecycle table and reuses existing feed, taxonomy, moderation, notifications and messaging infrastructure.

M.1 Product and scope decisions

- A Community Help item is a feed_posts row with post_type = COMMUNITY_HELP. The parent row remains the source of truth for author, community, country/audience, title/body, media, comments, reactions, publication, moderation and reports.
- There are two kinds: REQUEST (the author needs assistance) and OFFER (the author offers informal, unpaid assistance or donations). Categories can include Items & Donations, Practical Assistance, Language & Learning, Newcomer Support and Volunteering, configured per country.
- Community Help posts can appear in the dedicated Community Help section and optionally as normal cards in the main Feed. Do not create a second social graph, dedicated comment/reaction tables, payment workflow or separate chat system.
- Paid cleaning, babysitting, plumbing, tutoring or other professional services remain in servicemarketplace when charged as professional work. The Community Help feature is for informal assistance and volunteering.

M.2 Database changes

- Add COMMUNITY_HELP to feed_posts.post_type. All usual feed audience, content status, comments, reactions, media, blocks and moderation rules still apply. Create the post as DRAFT; do not directly publish a help post before its lifecycle row exists.
- Extend category_module with COMMUNITY_HELP. Administrators configure country_categories and map them in country_category_modules with module_key = COMMUNITY_HELP. validate_feed_post_country_category() uses the COMMUNITY_HELP module for this post type and FEED for other feed post types.
- Add community_help_posts as the only new feature table. post_id is both its primary key and FK to feed_posts.id. Fields: help_kind (REQUEST/OFFER), help_status (OPEN/PAUSED/RESOLVED/CLOSED/EXPIRED), optional expires_at, database-managed resolved_at/closed_at/expired_at and created/updated timestamps.
- Add COMMUNITY_HELP_POST to direct_conversations.context_type. context_id is the parent feed_posts.id, validated by the existing contextual conversation trigger. Existing participant-pair/context uniqueness is reused. No community_help_responses table is created.

M.3 Lifecycle and database integrity

- Creation workflow: insert feed_posts as DRAFT; insert community_help_posts with help_status = OPEN in the same transaction; then use the standard publication/moderation flow. The schema rejects publication without lifecycle metadata and rejects lifecycle metadata attached to a non-COMMUNITY_HELP parent.
- feed_posts.status controls publication/moderation. community_help_posts.help_status controls help availability. They are separate state dimensions.
- Supported transitions: OPEN ↔ PAUSED and OPEN/PAUSED → RESOLVED, CLOSED or EXPIRED. RESOLVED, CLOSED and EXPIRED are terminal. help_kind, post_id and author identity are immutable. resolved_at/closed_at/expired_at are set by the database on state changes, not directly by client payloads.
- A Spring Boot/Quartz scheduled task transitions due OPEN/PAUSED records to EXPIRED when expires_at <= now(). A NULL expires_at means no scheduled expiry. The active Community Help browse query returns only PUBLISHED, non-deleted parent posts whose help_status is OPEN and whose expiry is absent or in the future.
- A PAUSED or terminal help post is excluded from active Community Help discovery and cannot open new response conversations. If its parent remains published in the general feed, show the current help status and disable new responses. Existing conversations remain accessible subject to normal messaging permissions.

M.4 Contextual responses and messaging

- A member selects Respond/Contact to create or reuse a direct_conversations row with context_type = COMMUNITY_HELP_POST and context_id = feed_posts.id. The normal unique pair/context constraint prevents duplicate threads for the same pair and help post.
- A new contextual conversation is permitted only when the help parent exists, is published and is OPEN. The post author must be one of the conversation participants. Context-specific conversations do not use the GENERAL message-request acceptance workflow. The backend must enforce context-contact eligibility, audience visibility, blocks, privacy preferences, quotas, business identity and authorization on every relevant action.
- Send response notifications through the existing notification/outbox infrastructure when the new contextual conversation is created. Do not persist a duplicate response record; the conversation is the response channel. Closing a help item does not delete chat history.

M.5 Feed query and permissions

- Community Help query filters by help_kind, country/community category, visibility_scope, community membership, account status, blocks and moderation. Category is classification only and does not widen the content audience.
- The main Feed may render the same feed_posts row as a card. No duplicate post or media upload is created. Keep dedicated Community Help discovery focused on currently OPEN, unexpired items.
- Only the author can pause, reopen, resolve or close their help post, subject to moderator overrides. The backend validates state transitions. Moderators use existing hide/remove/report workflows on feed_posts and existing moderation audit records.

M.6 API contract candidates

- POST /api/v1/community-help — create the DRAFT parent post and OPEN lifecycle row in one transaction.
- POST /api/v1/community-help/{postId}/publish — publish after category, audience, content and moderation validation.
- GET /api/v1/community-help?kind=REQUEST|OFFER&categoryId=...&communityId=...&cursor=...&limit=... — browse OPEN, unexpired, visible items.
- POST /api/v1/community-help/{postId}/respond — create or reuse the contextual direct conversation, only for PUBLISHED/OPEN posts.
- PATCH /api/v1/community-help/{postId}/status — author transitions to PAUSED, OPEN (only from PAUSED), RESOLVED or CLOSED. Expiry is handled by a scheduled task, not by a client setting an arbitrary EXPIRED status.

M.7 Acceptance checks

1. Create a COMMUNITY_HELP post as DRAFT, insert lifecycle metadata, then publish. Publishing without the row must fail.

2. Reject lifecycle rows whose parent is not COMMUNITY_HELP; reject changing help_kind/post_id/author identity; reject deleting metadata from a published parent.

3. Verify country category mappings require module COMMUNITY_HELP for help posts and FEED for other feed post types.

4. Test state transitions, terminal-state immutability, managed timestamps, optional expiry and scheduled expiry.

5. Test that only PUBLISHED/OPEN targets create new contextual conversations, that the author is a participant, and that pair/context uniqueness reuses existing conversations.

6. Test visibility, blocks, moderation, notifications and existing conversation history after a help post is resolved/closed/expired. No community_help_responses table should exist.